

Electron Scientific Calculator — সম্পূর্ণ নির্মাণ গাইড

ভূমিকা

অধ্যায় ১ — Electron আর্কিটেকচার বোঝা

Main Process

Renderer Process

Preload Script

contextBridge কেন দরকার?

nodeIntegration কেন বন্ধ রাখা উচিত?

contextIsolation কেন গুরুত্বপূর্ণ?

যোগাযোগ কীভাবে কাজ করে (সারাংশ)

অধ্যায় ২ — প্রয়োজনীয় সফটওয়্যার

ইনস্টলেশন যাচাই করা

যদি Node.js/npm ইনস্টল করা না থাকে

অধ্যায় ৩ — Project Folder তৈরি করা

অধ্যায় ৪ — npm init দিয়ে প্রজেক্ট শুরু করা

package.json সংশোধন করা

অধ্যায় ৫ — Electron ইনস্টল করা

অধ্যায় ৬ — প্রজেক্ট স্ট্রাকচার তৈরি করা

অধ্যায় ৭ — Feature ১: Main Electron Window

অধ্যায় ৮ — Feature ২: Preload নিরাপত্তা

অধ্যায় ৯ — Feature ৩: বেসিক HTML ক্যালকুলেটর UI

অধ্যায় ১০ — Feature ৪: CSS ধাপে ধাপে তৈরি

CSS Stage ১ — Body, Container, Display

CSS Stage ২ — Keypad Grid

CSS Stage ৩ — Number, Operator, Function বাটন স্টাইল

CSS Stage ৪ — Hover, Active, Focus, Transition

CSS Stage ৫ — Responsive Sizing

অধ্যায় ১১ — Feature ৫: বেসিক ক্যালকুলেশন ইঞ্জিন

অধ্যায় ১২ — Feature ৬: বৈজ্ঞানিক ফাংশন (Scientific Features)
অধ্যায় ১৩ — Feature ৭: ত্রিকোণমিতি (Trigonometry) + DEG/RAD
অধ্যায় ১৪ — ক্যালকুলেটরের State Management ব্যাখ্যা
অধ্যায় ১৫ — Feature ৮: History System (localStorage)
অধ্যায় ১৬ — Feature ৯: History Drawer UI
অধ্যায় ১৭ — Feature ১০: ক্লিপবোর্ড সিস্টেম
অধ্যায় ১৮ — Feature ১১: Dark/Light থিম
অধ্যায় ১৯ — Feature ১২: Web Audio API দিয়ে সাউন্ড
অধ্যায় ২০ — Feature ১৩: কীবোর্ড সাপোর্ট
অধ্যায় ২১ — উইন্ডো কনফিগারেশন — সারসংক্ষেপ
অধ্যায় ২২ — এরর হ্যান্ডলিং — সম্পূর্ণ সারসংক্ষেপ
অধ্যায় ২৩ — সম্পূর্ণ টেস্টিং পরিকল্পনা

- মৌলিক গণিত
- বৈজ্ঞানিক
- ত্রিকোণমিতি (DEG মোডে)
- RAD মোড
- এজ কেস (Edge Cases)
- History
- থিম
- কীবোর্ড
- ক্লিপবোর্ড
- অডিও

অধ্যায় ২৪ — ডিবাগিং ও ট্রাবলশুটিং গাইড
অধ্যায় ২৫ — নিরাপত্তা (Security) নোট

- DO 
- DON'T 

অধ্যায় ২৬ — Do's এবং Don'ts (সাধারণ কোডিং অভ্যাস)

- DO 
- DON'T 

অধ্যায় ২৭ — ডেভেলপমেন্ট ওয়ার্কফ্লো (Git সহ)

- Git কখন ব্যবহার করবেন
- পরবর্তী কমিটগুলোর উদাহরণ

অধ্যায় ২৮ — চূড়ান্ত প্রজেক্ট স্ট্রাকচার
অধ্যায় ২৯ — চূড়ান্ত সম্পূর্ণ সোর্স কোড
package.json

main.js

preload.js

renderer/index.html

renderer/styles.css

renderer/renderer.js

অধ্যায় ৩০ — চূড়ান্ত রান নির্দেশনা

অধ্যায় ৩১ — প্যাকেজিং: ডিস্ট্রিবিউটেবল অ্যাপ তৈরি করা

কেন electron-packager দরকার

ইনস্টলেশন

কনফিগারেশন

Windows-এর জন্য বিল্ড করা

Linux-এর জন্য বিল্ড করা

অধ্যায় ৩২ — চূড়ান্ত প্রজেক্ট চেকলিস্ট

ভূমিকা

এই ডকুমেন্টটি একটি সম্পূর্ণ, হাতে-কলমে (hands-on) build manual। এখানে শূন্য থেকে শুরু করে একটি সম্পূর্ণ কার্যকরী **Electron Scientific Calculator** তৈরি করা হবে — ধাপে ধাপে, প্রতিটি অংশ ব্যাখ্যাসহ, প্রতিটি ফিচার আলাদাভাবে বানিয়ে টেস্ট করা হবে।

নিয়ম মনে রাখুন:

- ব্যাখ্যা (explanation) লেখা হয়েছে বাংলায়।
 - কোড, কমান্ড, ফাইলের নাম, ফোল্ডারের নাম সবসময় ইংরেজিতে থাকবে, ঠিক যেভাবে টাইপ করতে হবে।
 - প্রতিটি ফিচারের জন্য: লক্ষ্য → জড়িত ফাইল → কোড → কোথায় বসবে → কেন দরকার → রান করা → টেস্ট → প্রত্যাশিত ফলাফল → সাধারণ সমস্যা → সমাধান — এই কাঠামো অনুসরণ করা হয়েছে।
-

অধ্যায় ১ — Electron আর্কিটেকচার বোঝা

Electron দিয়ে ডেস্কটপ অ্যাপ বানানোর সময় দুইটি মূল প্রসেস (process) কাজ করে।

Main Process

Main Process হলো আমাদের অ্যাপ্লিকেশনের “ব্যাকস্টেজ ম্যানেজার”। এটি Node.js পরিবেশে চলে এবং এর কাজ হলো:

- অপারেটিং সিস্টেমের উইন্ডো (window) তৈরি করা (`BrowserWindow`)
- অ্যাপ্লিকেশনের লাইফসাইকেল (চালু হওয়া, বন্ধ হওয়া) নিয়ন্ত্রণ করা
- ফাইল সিস্টেম, নেটিভ মেনু, ইত্যাদির মতো সিস্টেম-লেভেল রিসোর্সে প্রবেশাধিকার থাকা

আমাদের প্রজেক্টে এটি থাকবে `main.js` ফাইলে।

Renderer Process

Renderer Process হলো সেই অংশ যেখানে আমাদের UI (HTML, CSS, JavaScript) চলে — অনেকটা একটি সাধারণ ওয়েব পেজের মতো, কিন্তু Chromium ব্রাউজার ইঞ্জিনের ভেতরে। ব্যবহারকারী যা দেখে এবং যেখানে ক্লিক করে, সব এখানেই ঘটে।

আমাদের প্রজেক্টে এটি থাকবে `renderer/index.html`, `renderer/styles.css`, এবং `renderer/renderer.js` ফাইলে।

Preload Script

নিরাপত্তার কারণে, Renderer Process থেকে সরাসরি Node.js-এর শক্তিশালী API (যেমন ফাইল সিস্টেম, ক্লিপবোর্ড) ব্যবহার করতে দেওয়া হয় না। এর বদলে একটি **preload script** ব্যবহার করা হয়, যেটি Main এবং Renderer-এর মাঝে একটি নিয়ন্ত্রিত, নিরাপদ সেতু (bridge) হিসেবে কাজ করে।

আমাদের প্রজেক্টে এটি থাকবে `preload.js` ফাইলে।

contextBridge কেন দরকার?

`contextBridge` হলো Electron-এর একটি API যা preload script-কে নির্দিষ্ট কিছু ফাংশন Renderer-এর `window` অবজেক্টে “এক্সপোজ” (expose) করতে দেয় — কিন্তু পুরো Node.js API নয়, শুধু যা আমরা ইচ্ছাকৃতভাবে দিতে চাই। এটি অনেকটা রিসেপশন কাউন্টারের মতো: ভেতরের পুরো অফিসে ঢোকার অনুমতি নেই, কিন্তু নির্দিষ্ট কিছু ফর্ম রিসেপশনের মাধ্যমে জমা দেওয়া যায়।

nodeIntegration কেন বন্ধ রাখা উচিত?

`nodeIntegration: true` করলে Renderer Process-এর ভেতরের যেকোনো JavaScript কোড (এমনকি যদি ভুলবশত কোনো ক্ষতিকর কোড HTML-এ ঢুকে যায়) সরাসরি Node.js-এর ফাইল সিস্টেম, প্রসেস কন্ট্রোল ইত্যাদি ব্যবহার করতে পারবে। এটি একটি গুরুতর নিরাপত্তা ঝুঁকি। তাই আমরা সবসময় `nodeIntegration: false` রাখব।

contextIsolation কেন গুরুত্বপূর্ণ?

`contextIsolation: true` থাকলে preload script এবং renderer-এর JavaScript আলাদা “জগতে” (context) চলে। ফলে renderer-এর কোনো কোড preload-এর ভেরিয়েবল বা Node.js অবজেক্টে সরাসরি হাত দিতে পারে না — শুধুমাত্র `contextBridge` দিয়ে ইচ্ছাকৃতভাবে এক্সপোজ করা ফাংশনগুলোই ব্যবহার করতে পারে।

যোগাযোগ কীভাবে কাজ করে (সারাংশ)

```
Renderer (index.html, renderer.js)
  | window.electronAPI.copyToClipboard(...)
  ▼
Preload (preload.js) – contextBridge দিয়ে সংযুক্ত
  | clipboard.writeText(...)
  ▼
Electron / Main Process – প্রকৃত সিস্টেম API চালায়
```

আমাদের ক্যালকুলেটরে আমরা এই প্যাটার্নটি ব্যবহার করব ক্লিপবোর্ড কপি-পেস্ট ফিচারের জন্য।

অধ্যায় ২ — প্রয়োজনীয় সফটওয়্যার

শুরু করার আগে নিচের জিনিসগুলো লাগবে:

প্রয়োজন	ব্যাখ্যা
Node.js	JavaScript রান করার রানটাইম। Electron আসলে Node.js এবং Chromium-এর সমন্বয়।
npm	Node.js-এর সাথেই আসে। এটি প্যাকেজ ম্যানেজার — লাইব্রেরি (যেমন electron) ইনস্টল করতে ব্যবহার হয়।
কোড এডিটর	VS Code ব্যবহার করা সবচেয়ে সুবিধাজনক, তবে যেকোনো এডিটর চলবে।
টার্মিনাল	কমান্ড রান করার জন্য (Windows-এ PowerShell/CMD, Linux/macOS-এ Terminal)।

ইনস্টলেশন যাচাই করা

টার্মিনাল খুলে নিচের কমান্ড দুটি রান করুন:

```
node --version  
npm --version
```

প্রত্যাশিত ফলাফল: প্রতিটি কমান্ড একটি ভার্সন নাম্বার দেখাবে, যেমন:

```
v20.11.0  
10.2.4
```

যদি সংখ্যা প্রদর্শিত হয়, তাহলে Node.js এবং npm সঠিকভাবে ইনস্টল করা আছে।

যদি Node.js/npm ইনস্টল করা না থাকে

1. <https://nodejs.org> থেকে **LTS (Long Term Support)** ভার্সনটি ডাউনলোড করুন।
2. ইনস্টলার চালিয়ে ডিফল্ট সেটিংস দিয়ে ইনস্টল করুন।

3. ইনস্টলেশনের পর টার্মিনাল/কমান্ড প্রম্পট **বন্ধ করে আবার খুলুন** (নতুন PATH ভ্যারিয়েবল লোড হওয়ার জন্য এটি জরুরি)।
4. উপরের ভেরিফিকেশন কমান্ড দুটি আবার চালিয়ে নিশ্চিত হোন।

সাধারণ সমস্যা: `node: command not found` বা `'node' is not recognized`
— এর অর্থ Node.js ইনস্টল হয়নি অথবা PATH-এ যোগ হয়নি। ইনস্টলার আবার চালান এবং টার্মিনাল রিস্টার্ট করুন।

অধ্যায় ৩ — Project Folder তৈরি করা

টার্মিনালে নিচের কমান্ড দুটি চালান:

```
mkdir electron-scientific-calculator  
cd electron-scientific-calculator
```

ব্যাখ্যা:

- `mkdir electron-scientific-calculator` — এই কমান্ডটি বর্তমান লোকেশনে `electron-scientific-calculator` নামে একটি নতুন ফোল্ডার (directory) তৈরি করে। এখানেই আমাদের সম্পূর্ণ প্রজেক্ট থাকবে।
- `cd electron-scientific-calculator` — `cd` (change directory) কমান্ডটি আমাদের টার্মিনালকে সেই নতুন ফোল্ডারের ভেতরে নিয়ে যায়, যাতে এরপর থেকে আমরা যা কিছু কমান্ড চালাই, সব এই ফোল্ডারের ভেতরেই ঘটে।

সতর্কতা: ফোল্ডারের নামে স্পেস বা বাংলা অক্ষর ব্যবহার করবেন না — এতে পরবর্তীতে বিভিন্ন টুলে সমস্যা হতে পারে।

টেস্ট: `pwd` (Linux/macOS) অথবা `cd` শুধু (Windows) চালিয়ে দেখুন বর্তমান পথের শেষে `electron-scientific-calculator` দেখাচ্ছে কিনা।

অধ্যায় ৪ — npm init দিয়ে প্রজেক্ট শুরু করা

নিচের কমান্ডটি চালান:

```
npm init -y
```

ব্যাখ্যা: এই কমান্ডটি একটি `package.json` ফাইল তৈরি করে, যেটি প্রতিটি Node.js/Electron প্রজেক্টের “পরিচয়পত্র”। `-y` ফ্ল্যাগ দেওয়া হলে npm প্রতিটি প্রশ্ন না করে ডিফল্ট মান দিয়ে সরাসরি ফাইলটি তৈরি করে দেয়।

`package.json`-এর গুরুত্বপূর্ণ ফিল্ডগুলো:

- **name** — প্রজেক্টের নাম।
- **version** — প্রজেক্টের ভার্সন নম্বর।
- **main** — যে ফাইল থেকে অ্যাপ্লিকেশন শুরু হবে (আমাদের ক্ষেত্রে `main.js`)।
- **scripts** — শর্টকাট কমান্ড, যেমন `npm start` চালালে কী রান হবে তা এখানে বলা থাকে।
- **dependencies** — যেসব লাইব্রেরি অ্যাপ্লিকেশন চলার জন্য দরকার (production-এ প্রয়োজন হয়)।
- **devDependencies** — যেসব লাইব্রেরি শুধু ডেভেলপমেন্টের সময় দরকার, যেমন `electron` নিজেই।

`package.json` সংশোধন করা

`package.json` ফাইলটি এডিটরে খুলে নিচের মতো করে সাজান:

```
{
  "name": "electron-scientific-calculator",
  "version": "1.0.0",
  "description": "A scientific calculator built with Electron",
  "main": "main.js",
  "scripts": {
    "start": "electron .",
    "package-win": "electron-packager . ScientificCalculator --platform=win32 --arch=x64 --out=dist --overwrite",
    "package-linux": "electron-packager . ScientificCalculator --platform=linux --arch=x64 --out=dist --overwrite"
  },
  "author": "",
```

```
"license": "ISC",  
"devDependencies": {  
  "electron": "^31.0.0",  
  "electron-packager": "^17.1.2"  
}  
}
```

কোথায় বসবে: এই সম্পূর্ণ কোডটি `electron-scientific-calculator/package.json` ফাইলের ভেতরে থাকবে, ইতিমধ্যে `npm init -y` যা তৈরি করেছে তা প্রতিস্থাপন করে।

কেন দরকার: `"main": "main.js"` বলে দিচ্ছে Electron কোন ফাইল থেকে শুরু করবে। `scripts.start` দিয়ে আমরা পরে শুধু `npm start` লিখেই অ্যাপ চালু করতে পারব।

অধ্যায় ৫ — Electron ইনস্টল করা

নিচের কমান্ডটি চালান:

```
npm install --save-dev electron electron-packager
```

ব্যাখ্যা: `--save-dev` ফ্ল্যাগ দিয়ে `electron` এবং `electron-packager` লাইব্রেরি দুটি `devDependencies`-তে যোগ হয়। Electron সাধারণত dev dependency হিসেবে ইনস্টল করা হয় কারণ এটি শুধু ডেভেলপমেন্ট এবং প্যাকেজিং-এর সময় দরকার হয় — চূড়ান্ত ব্যবহারকারীর কাছে যখন আমরা `.exe` বা প্যাকেজড অ্যাপ পাঠাই, তখন Electron রানটাইম প্যাকেজের ভেতরেই বান্ডিল হয়ে যায়, আলাদাভাবে `npm install` করার দরকার পড়ে না।

প্রত্যাশিত ফলাফল: ইনস্টলেশন শেষে একটি `node_modules` ফোল্ডার এবং `package-lock.json` ফাইল তৈরি হবে। কমান্ডের শেষে কিছু সংখ্যক প্যাকেজ ইনস্টল হওয়ার বার্তা দেখাবে।

সাধারণ সমস্যা: ইনস্টলেশন অনেক সময় লাগলে বা ব্যর্থ হলে, ইন্টারনেট সংযোগ পরীক্ষা করুন এবং আবার চেষ্টা করুন। কর্পোরেট নেটওয়ার্কে থাকলে npm-এর proxy সেটিং ঠিক করার প্রয়োজন হতে পারে।

অধ্যায় ৬ — প্রজেক্ট স্ট্রাকচার তৈরি করা

এখন নিচের ফাইল/ফোল্ডারগুলো তৈরি করব:

```
electron-scientific-calculator/
├── package.json          (আগেই তৈরি)
├── main.js
├── preload.js
├── renderer/
│   ├── index.html
│   ├── styles.css
│   └── renderer.js
└── node_modules/        (npm install স্বয়ংক্রিয়ভাবে তৈরি করেছে)
```

টার্মিনাল থেকে খালি ফাইলগুলো তৈরি করতে:

```
touch main.js preload.js
mkdir renderer
touch renderer/index.html renderer/styles.css renderer/
renderer.js
```

(Windows-এ PowerShell ব্যবহার করলে `touch` এর বদলে `New-Item` ব্যবহার করুন, অথবা এডিটরে সরাসরি “New File” দিয়ে ফাইল তৈরি করুন।)

ব্যাখ্যা:

- `main.js` — Main Process কোড।
- `preload.js` — নিরাপদ ব্রিজ কোড।
- `renderer/index.html` — ক্যালকুলেটরের HTML কাঠামো।
- `renderer/styles.css` — ক্যালকুলেটরের ডিজাইন।
- `renderer/renderer.js` — ক্যালকুলেটরের সমস্ত লজিক (হিসাব, ইতিহাস, থিম, সাউন্ড, কীবোর্ড)।

Renderer-এর সব ফাইল আলাদা `renderer/` ফোল্ডারে রাখা হয়েছে যাতে Main Process কোড এবং UI কোড স্পষ্টভাবে আলাদা থাকে — এটি বড় প্রজেক্টে কোড সংগঠিত রাখার একটি সাধারণ চর্চা।

টেষ্ট: এই ধাপ শেষে আপনার এডিটরের ফাইল এক্সপ্লোরারে উপরের কাঠামোর সাথে মিলিয়ে দেখুন।

অধ্যায় ৭ — Feature ১: Main Electron Window

১. Goal (লক্ষ্য)

একটি কার্যকরী Electron উইন্ডো তৈরি করা, যেটি নিরাপদ সেটিংস ব্যবহার করে আমাদের `index.html` লোড করবে।

২. জড়িত ফাইল

- `main.js`

৩. Implementation (কোড)

```
const { app, BrowserWindow, Menu } = require('electron');
const path = require('path');

function createWindow() {
  const mainWindow = new BrowserWindow({
    width: 380,
    height: 680,
    minWidth: 340,
    minHeight: 560,
    title: 'Scientific Calculator',
    webPreferences: {
      preload: path.join(__dirname, 'preload.js'),
      contextIsolation: true,
      nodeIntegration: false
    }
  });

  Menu.setApplicationMenu(null);

  mainWindow.loadFile(path.join(__dirname, 'renderer',
    'index.html'));
}

app.whenReady().then(() => {
  createWindow();

  app.on('activate', () => {
    if (BrowserWindow.getAllWindows().length === 0) {
```

```

        createWindow();
    }
});
});

app.on('window-all-closed', () => {
    if (process.platform !== 'darwin') {
        app.quit();
    }
});

```

৪. কোথায় বসবে

পুরো কোডটি `electron-scientific-calculator/main.js` ফাইলে বসবে। এটিই একমাত্র কোড যা এই ফাইলে থাকবে।

৫. কেন এই কোড দরকার (ব্যাখ্যা)

- `BrowserWindow` — Electron-এর ক্লাস যা দিয়ে আসল অপারেটিং সিস্টেম উইন্ডো তৈরি হয়।
- `width / height` — উইন্ডোর প্রাথমিক আকার (পিক্সেলে)। ক্যালকুলেটরের জন্য একটি সরু, লম্বা উইন্ডো (৩৮০x৬৮০) উপযুক্ত, কারণ ফোনের ক্যালকুলেটরের মতো দেখতে চাই।
- `minWidth / minHeight` — ব্যবহারকারী উইন্ডো ছোট করলেও এর চেয়ে ছোট হতে পারবে না, যাতে বাটনগুলো ভেঙে না যায়।
- `title` — উইন্ডোর টাইটেল বারে যে লেখা দেখাবে।
- `webPreferences.preload` — কোন preload script ব্যবহার হবে তার সম্পূর্ণ পথ। `path.join(__dirname, 'preload.js')` ব্যবহার করা হয়েছে যাতে Windows/Linux/macOS — সব প্ল্যাটফর্মেই পথ সঠিকভাবে তৈরি হয়।
- `contextIsolation: true` এবং `nodeIntegration: false` — আগেই ব্যাখ্যা করা নিরাপত্তা সেটিংস।
- `Menu.setApplicationMenu(null)` — Electron-এর ডিফল্ট মেনু বার (File, Edit, View...) লুকিয়ে দেয়, কারণ একটি ক্যালকুলেটরে এসবের দরকার নেই।
- `mainWindow.loadFile(...)` — আমাদের HTML ফাইলটি উইন্ডোর ভেতরে লোড করে।
- `app.whenReady()` — Electron-এর অভ্যন্তরীণ প্রস্তুতি শেষ হলে তবেই উইন্ডো তৈরি হয়।
- `app.on('activate', ...)` — macOS-এ ডক আইকনে ক্লিক করলে উইন্ডো না থাকলে আবার তৈরি করে (macOS আচরণ অনুযায়ী প্রয়োজনীয়)।

- `app.on('window-all-closed', ...)` — সব উইন্ডো বন্ধ হলে অ্যাপ বন্ধ করে দেয়, তবে macOS-এ ডিফল্ট আচরণ অনুযায়ী অ্যাপ চালু থাকে (`darwin` বাদে)।

৬. রান করা

এই মুহূর্তে `preload.js` এবং `index.html` খালি থাকায় সরাসরি রান করলে খালি সাদা উইন্ডো দেখাবে বা এরর আসতে পারে — এটাই স্বাভাবিক, কারণ আমরা এখনো সব ফাইল পূরণ করিনি। আপাতত রান কমান্ডটি জেনে রাখুন:

```
npm start
```

পরবর্তী ফিচারগুলো যোগ করার পর আমরা বারবার এই কমান্ডটি ব্যবহার করব।

৭. Testing

এখনই রান না করে পরের দুই ফিচার (`preload` + `HTML`) শেষ করার পর একসাথে টেস্ট করাই ভালো।

৮. প্রত্যাশিত ফলাফল

পরবর্তী ধাপ শেষে `npm start` চালালে একটি `৩৮০x৬৮০` আকারের উইন্ডো খুলবে, টাইটেল বারে “Scientific Calculator” লেখা থাকবে, এবং কোনো মেনু বার দেখাবে না।

৯. সাধারণ সমস্যা

- **সাদা/খালি উইন্ডো:** `index.html` এখনো খালি — স্বাভাবিক এই পর্যায়ে।
- **Cannot find module 'electron':** `npm install --save-dev electron` কমান্ডটি ঠিকভাবে চলেনি। আবার চালান।

১০. সমাধান

`node_modules` ফোল্ডার আছে কিনা যাচাই করুন। না থাকলে অধ্যায় ৫-এর কমান্ড আবার চালান।

অধ্যায় ৮ — Feature ২: Preload নিরাপত্তা

১. Goal

Renderer-কে নিয়ন্ত্রিতভাবে ক্লিপবোর্ড ব্যবহারের অনুমতি দেওয়া, সম্পূর্ণ Node.js API না দিয়ে।

২. জড়িত ফাইল

- `preload.js`

৩. Implementation

```
const { contextBridge, clipboard } = require('electron');

contextBridge.exposeInMainWorld('electronAPI', {
  copyToClipboard: (text) => clipboard.writeText(text),
  readFromClipboard: () => clipboard.readText()
});
```

৪. কোথায় বসবে

পুরো কোডটি `electron-scientific-calculator/preload.js` ফাইলে বসবে।

৫. কেন এই কোড দরকার

- `contextBridge.exposeInMainWorld('electronAPI', {...})` —
Renderer-এর `window` অবজেক্টে `window.electronAPI` নামে একটি নতুন বস্তু যোগ করে, যার ভেতরে শুধু আমরা যে দুটি ফাংশন দিয়েছি (`copyToClipboard`, `readFromClipboard`) সেগুলোই ব্যবহারযোগ্য থাকবে।
- **কী এক্সপোজ করা উচিত:** শুধু নির্দিষ্ট, ছোট পরিসরের ফাংশন — যেমন ক্লিপবোর্ড পড়া/লেখা।
- **কী এক্সপোজ করা উচিত নয়:** সরাসরি `require('fs')`, `require('child_process')` বা পুরো `clipboard` মডিউল কখনোই সরাসরি এক্সপোজ করা উচিত নয় — কারণ তাহলে Renderer-এর যেকোনো কোড সিস্টেমে সরাসরি প্রবেশাধিকার পেয়ে যাবে, যা নিরাপত্তা ঝুঁকি তৈরি করে।

৬. রান করা

এখনো `npm start` চালানোর দরকার নেই — HTML ফাইল খালি।

৭. Testing

পরের ফিচারের পর একত্রে টেস্ট করা হবে।

৮. প্রত্যাশিত ফলাফল

কোনো এরর ছাড়াই অ্যাপ চালু হবে (HTML যোগ করার পর)।

৯. সাধারণ সমস্যা

- **contextBridge is not defined:** `require('electron')` থেকে সঠিকভাবে ইমপোর্ট করা হয়নি কিনা যাচাই করুন।
- **window.electronAPI is undefined (renderer.js-এ পরে দেখা যেতে পারে):** `main.js`-এ `preload` পাথ ভুল থাকলে এমন হয়। `path.join(__dirname, 'preload.js')` ঠিক আছে কিনা যাচাই করুন।

১০. সমাধান

`preload` ফাইলের পাথ এবং ফাইলের নাম হুবহু মিলিয়ে দেখুন (case-sensitive — বড়/ছোট হাতের অক্ষর গুরুত্বপূর্ণ, বিশেষত Linux-এ)।

অধ্যায় ৯ — Feature ৩: বেসিক HTML ক্যালকুলেটর UI

১. Goal

সংখ্যা বাটন, অপারেটর বাটন, একটি ডিসপ্লে এবং একটি হিস্টোরি প্যানেলসহ ক্যালকুলেটরের কাঠামো তৈরি করা। এখনই সব বৈজ্ঞানিক ফাংশন যোগ করছি না — আগে বেসিক কাঠামো কাজ করছে কিনা তা নিশ্চিত করব।

২. জড়িত ফাইল

- `renderer/index.html`

৩. Implementation

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>Scientific Calculator</title>
  <link rel="stylesheet" href="styles.css" />
</head>
<body>
  <div class="app" id="app">
    <div class="top-bar">
      <button id="themeToggle" class="icon-btn" title="Toggle
theme">🌙</button>
      <span class="angle-mode" id="angleMode">DEG</span>
      <button id="muteToggle" class="icon-btn" title="Mute">🔊</button>
      <button id="historyToggle" class="icon-btn"
title="History">📜</button>
    </div>

    <div class="display">
      <div class="prev-operand" id="prevOperand"></div>
      <div class="current-operand" id="currentOperand">0</div>
    </div>

    <div class="keypad">
      <button class="btn function" data-action="clear">C</button>
    </div>
  </div>
</body>
</html>
```

```

        <button class="btn function" data-action="deg-rad">DEG</button>
        <button class="btn function" data-action="sqrt"> $\sqrt{x}$ </button>
        <button class="btn operator" data-action="divide"> $\div$ </button>

        <button class="btn function" data-action="square"> $x^2$ </button>
        <button class="btn function" data-action="power"> $x^y$ </button>
        <button class="btn function" data-action="log">log</button>
        <button class="btn operator" data-action="multiply"> $\times$ </button>

        <button class="btn number" data-number="7">7</button>
        <button class="btn number" data-number="8">8</button>
        <button class="btn number" data-number="9">9</button>
        <button class="btn operator" data-action="subtract"> $-$ </button>

        <button class="btn number" data-number="4">4</button>
        <button class="btn number" data-number="5">5</button>
        <button class="btn number" data-number="6">6</button>
        <button class="btn operator" data-action="add"> $+$ </button>

        <button class="btn number" data-number="1">1</button>
        <button class="btn number" data-number="2">2</button>
        <button class="btn number" data-number="3">3</button>
        <button class="btn function" data-action="sin">sin</button>

        <button class="btn number zero" data-number="0">0</button>
        <button class="btn number" data-number=".">.</button>
        <button class="btn function" data-action="cos">cos</button>
        <button class="btn function" data-action="tan">tan</button>

        <button class="btn function" data-action="pi"> $\pi$ </button>
        <button class="btn function" data-action="copy">copy</button>
        <button class="btn equals" data-action="equals"> $=$ </button>
    </div>
</div>

<div class="history-panel" id="historyPanel">

```

```

<div class="history-header">
  <span>History</span>
  <button id="closeHistory" class="icon-btn">x</button>
</div>
<div class="history-list" id="historyList"></div>
<button id="clearHistory" class="btn function full-
width">Clear History</button>
</div>

<script src="renderer.js"></script>
</body>
</html>

```

৪. কোথায় বসবে

পুরো কোডটি `electron-scientific-calculator/renderer/index.html` ফাইলে বসবে।

৫. কেন এই কোড দরকার

- **data-number** এবং **data-action attribute**: প্রতিটি বাটনে আলাদা `id` না দিয়ে আমরা `data-*` attribute ব্যবহার করেছি। এতে `renderer.js`-এ আমরা এক লাইনে সব সংখ্যা বাটন বা সব অ্যাকশন বাটনকে একসাথে খুঁজে বের করে ইভেন্ট যোগ করতে পারব — কোড অনেক ছোট ও পরিষ্কার থাকে।
- **prevOperand / currentOperand**: দুটি আলাদা ডিসপ্লে লাইন — উপরেরটায় আগের সংখ্যা+অপারেটর দেখাবে (যেমন `12 +`), নিচেরটায় বর্তমান ইনপুট/ফলাফল।
- **historyPanel**: এটি একটি লুকানো সাইড প্যানেল যা পরে CSS দিয়ে স্লাইড-ইন করানো হবে।
- **<script src="renderer.js">**: `</body>`-এর ঠিক আগে রাখা হয়েছে, যাতে স্ক্রিপ্ট চলার আগে সব HTML এলিমেন্ট তৈরি হয়ে যায় (`document.getElementById` কল করার সময় এলিমেন্ট যেন `null` না আসে)।

৬. রান করা

```
npm start
```

৭. Testing

উইন্ডো খুলে দেখুন:

1. ক্যালকুলেটরের কাঠামো (ডিসপ্লে + বাটন গ্রিড) দেখা যাচ্ছে কিনা।
2. বাটনে ক্লিক করলে এখনো কিছু ঘটবে না — এটাই স্বাভাবিক, কারণ `renderer.js` এখনো খালি।

৮. প্রত্যাশিত ফলাফল

স্টাইল ছাড়া (unstyled) হলেও, ব্রাউজারের ডিফল্ট বাটন-স্টাইলে সব বাটন এবং ডিসপ্লে দেখা যাবে।

৯. সাধারণ সমস্যা

- **Blank window / কিছুই দেখা যাচ্ছে না:** `mainWindow.loadFile` পাথ ভুল থাকলে এমন হয়। `main.js` -এ পাথ `renderer/index.html` ঠিক আছে কিনা দেখুন।
- **DevTools খুলতে চাইলে:** সাময়িকভাবে `main.js` -এ `mainWindow.webContents.openDevTools();` যোগ করে এরর দেখতে পারেন (পরে এটি সরিয়ে ফেলুন)।

১০. সমাধান

ফাইলের নাম ও ফোল্ডারের নাম (case-sensitive) আবার যাচাই করুন।

অধ্যায় ১০ — Feature 8: CSS ধাপে ধাপে তৈরি

CSS একবারে পুরোটা না লিখে ধাপে ধাপে তৈরি করা হবে, যাতে প্রতিটি অংশ কী পরিবর্তন করছে তা স্পষ্ট বোঝা যায়। প্রতিটি Stage-এর কোড ধারাবাহিকভাবে `renderer/styles.css` ফাইলে **যোগ** হতে থাকবে (আগের Stage-এর কোড মুছে ফেলবেন না)।

CSS Stage ১ — Body, Container, Display

```
:root {
  --bg: #1e1e2e;
  --app-bg: #292a3e;
  --display-bg: #20212f;
  --text: #f4f4f8;
  --btn-number-bg: #3a3b52;
  --btn-function-bg: #4b4d6b;
  --btn-operator-bg: #f0883e;
  --btn-equals-bg: #4caf7d;
  --btn-active: #5a5c7f;
  --border: #45465e;
}

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  background: var(--bg);
  font-family: 'Segoe UI', Arial, sans-serif;
  color: var(--text);
  display: flex;
  justify-content: center;
  transition: background 0.25s ease, color 0.25s ease;
}

.app {
  width: 100%;
  max-width: 380px;
  background: var(--app-bg);
  padding: 16px;
  border-radius: 12px;
  margin-top: 16px;
  transition: background 0.25s ease;
```

```

}

.display {
  background: var(--display-bg);
  border-radius: 8px;
  padding: 16px 12px;
  text-align: right;
  margin-bottom: 12px;
  min-height: 72px;
  transition: background 0.25s ease;
}

.prev-operand {
  font-size: 14px;
  opacity: 0.6;
  min-height: 18px;
}

.current-operand {
  font-size: 32px;
  word-wrap: break-word;
  word-break: break-all;
}

```

ব্যাখ্যা (বাংলা): `:root` এর ভেতরে CSS **custom properties** (variable) সংজ্ঞায়িত করা হয়েছে — যেমন `--bg`, `--text`। এগুলো পরে থিম (dark/light) পরিবর্তনের সময় কাজে লাগবে (Stage-এর বাইরে, অধ্যায় ১৮-এ)। `box-sizing: border-box` দেওয়া হয়েছে যাতে padding যোগ করলে এলিমেন্টের প্রকৃত প্রস্থ না বেড়ে যায় — এটি লেআউট হিসাব সহজ রাখে।

রান করুন এবং দেখুন: `npm start` — এখন অ্যাপ একটি অন্ধকার (dark) ব্যাকগ্রাউন্ডযুক্ত কার্ডের মতো দেখাবে, উপরে একটি ডিসপ্লে বক্স থাকবে (এখনো বাটনগুলো সাজানো নয়)।

টেষ্ট: ডিসপ্লে বক্সে ডানদিকে “0” লেখা দেখা উচিত।

CSS Stage ২ — Keypad Grid

```

.keypad {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 8px;
}

```

ব্যাখ্যা: CSS Grid ব্যবহার করে বাটনগুলোকে ৪-কলামের একটি গ্রিডে সাজানো হচ্ছে। `gap: 8px` প্রতিটি বাটনের মাঝে সমান ফাঁক রাখে।

রান করে দেখুন: বাটনগুলো এখন ৪-কলামের গ্রিডে সাজানো দেখাবে, কিন্তু বাটনগুলোর নিজস্ব রঙ এখনো ব্রাউজার ডিফল্ট।

CSS Stage ৩ — Number, Operator, Function বাটন স্টাইল

```
.btn {
  border: none;
  border-radius: 8px;
  padding: 16px 0;
  font-size: 16px;
  cursor: pointer;
  color: var(--text);
  background: var(--btn-number-bg);
  transition: background 0.15s ease, transform 0.05s ease;
}

.btn.function {
  background: var(--btn-function-bg);
}

.btn.operator {
  background: var(--btn-operator-bg);
  color: #fff;
}

.btn.equals {
  background: var(--btn-equals-bg);
  color: #fff;
  grid-column: span 2;
}

.btn.zero {
  grid-column: span 1;
}
```

ব্যাখ্যা: বেস `.btn` ক্লাস সব বাটনের সাধারণ স্টাইল দেয় (গোল কোণা, কার্সর পয়েন্টার ইত্যাদি)। এরপর `.function`, `.operator`, `.equals` — প্রতিটি ভিন্ন ক্যাটাগরির বাটনকে ভিন্ন রঙ দেয়, যাতে ব্যবহারকারী দৃষ্টিগতভাবে সহজে বুঝতে পারে কোনটা সংখ্যা, কোনটা অপারেটর। `grid-column: span 2` দিয়ে `=` বাটনটি দুই কলাম জুড়ে বড় করা হয়েছে, কারণ এটি সবচেয়ে বেশি ব্যবহৃত বাটন।

রান করে দেখুন: এখন প্রতিটি বাটনের রঙ আলাদা — সংখ্যা বেগুনি-ধূসর, অপারেটর কমলা, `=` সবুজ ও বড়।

CSS Stage 8 — Hover, Active, Focus, Transition

```
.btn:hover {  
  filter: brightness(1.08);  
}  
  
.btn:active,  
.btn.pressed {  
  background: var(--btn-active);  
  transform: scale(0.96);  
}
```

ব্যাখ্যা: `:hover` মাউস বাটনের উপর গেলে সামান্য উজ্জ্বল করে — এটি ব্যবহারকারীকে ইঙ্গিত দেয় যে বাটনটি ক্লিকযোগ্য। `:active` ক্লিক করার মুহুর্তে বাটনটি সামান্য ছোট করে (`scale(0.96)`) — এতে বাস্তব বাটন চাপার অনুভূতি হয়। `.pressed` ক্লাসটি আমরা পরে `renderer.js`-এ কীবোর্ড দিয়ে বাটন চাপার ভিজুয়াল ইফেক্ট দেখানোর জন্য ব্যবহার করব (অধ্যায় ২০)।

CSS Stage ৫ — Responsive Sizing

```
@media (max-width: 400px) {  
  .app {  
    max-width: 100%;  
    border-radius: 0;  
  }  
}
```

ব্যাখ্যা: `@media` কুয়েরি দিয়ে ছোট স্ক্রিনে (৪০০ পিক্সেলের কম প্রস্থ) `.app` কন্টেইনারকে পুরো স্ক্রিন জুড়ে প্রসারিত করে দেওয়া হয়, এবং কোণা গোল না রেখে সোজা রাখা হয় — এতে ছোট উইন্ডোতেও ক্যালকুলেটরটি সুন্দর দেখায়।

সম্পূর্ণ Stage ১-৫ টেস্ট: `npm start` চালিয়ে উইন্ডোর আকার ছোট-বড় করে দেখুন লেআউট ভেঙে যাচ্ছে কিনা। বাটনে মাউস নিয়ে গিয়ে/ক্লিক করে হোভার ও প্রেস ইফেক্ট পরীক্ষা করুন।

সাধারণ সমস্যা: CSS পরিবর্তন দেখা না গেলে — Electron উইন্ডোতে ফাইল পরিবর্তনের পর রিলোড হয় না নিজে থেকে। উইন্ডো বন্ধ করে `npm start` আবার চালান, অথবা DevTools খুলে (Ctrl+Shift+I / Cmd+Option+I) `Ctrl+R` দিয়ে রিলোড করুন।

অধ্যায় ১১ — Feature ৫: বেসিক ক্যালকুলেশন ইঞ্জিন

১. Goal

যোগ, বিয়োগ, গুণ, ভাগ — এই চারটি মৌলিক গাণিতিক অপারেশন কাজ করানো, সাথে Clear এবং দশমিক (decimal) ইনপুট ঠিকভাবে হ্যান্ডেল করা।

২. জড়িত ফাইল

- `renderer/renderer.js`

৩. Implementation

```
// ===== STATE =====
let currentInput = '0';
let previousInput = '';
let operator = null;
let resultJustShown = false;

const currentOperandEl =
document.getElementById('currentOperand');
const prevOperandEl = document.getElementById('prevOperand');

// ===== CALCULATION =====
function calculate(a, op, b) {
  a = parseFloat(a);
  b = parseFloat(b);
  switch (op) {
    case '+': return a + b;
    case '-': return a - b;
    case '*': return a * b;
    case '/': return b === 0 ? 'Error' : a / b;
    default: return b;
  }
}

// ===== DISPLAY =====
function updateDisplay() {
  currentOperandEl.textContent = currentInput;
  prevOperandEl.textContent = previousInput && operator ? ` $
{previousInput} ${operator}` : '';
}
```

```

}

function setError() {
  currentInput = 'Error';
  previousInput = '';
  operator = null;
  resultJustShown = true;
  updateDisplay();
}

// ===== INPUT HANDLERS =====
function inputNumber(num) {
  if (currentInput === 'Error' || resultJustShown) {
    currentInput = '';
    resultJustShown = false;
  }
  if (num === '.' && currentInput.includes('.')) return;
  if (currentInput === '0' && num !== '.') {
    currentInput = num;
  } else {
    currentInput = currentInput === '' ? num : currentInput +
num;
  }
  updateDisplay();
}

function chooseOperator(op) {
  if (currentInput === 'Error') return;
  if (previousInput !== '' && operator && !resultJustShown) {
    previousInput = calculate(previousInput, operator,
currentInput).toString();
  } else {
    previousInput = currentInput;
  }
  operator = op;
  currentInput = '0';
  resultJustShown = false;
  updateDisplay();
}

function equals() {
  if (operator === null || currentInput === 'Error') return;
  const result = calculate(previousInput, operator,
currentInput);
  if (result === 'Error') {
    setError();
    return;
  }
  currentInput = result.toString();
  previousInput = '';
}

```

```

    operator = null;
    resultJustShown = true;
    updateDisplay();
  }

  function clearAll() {
    currentInput = '0';
    previousInput = '';
    operator = null;
    resultJustShown = false;
    updateDisplay();
  }

  // ===== BUTTON WIRING =====
  document.querySelectorAll('.btn[data-number]').forEach((btn) =>
  {
    btn.addEventListener('click', () =>
    inputNumber(btn.dataset.number));
  });

  const actionMap = {
    clear: clearAll,
    equals: equals,
    add: () => chooseOperator('+'),
    subtract: () => chooseOperator('-'),
    multiply: () => chooseOperator('*'),
    divide: () => chooseOperator('/')
  };

  document.querySelectorAll('.btn[data-action]').forEach((btn) =>
  {
    btn.addEventListener('click', () => {
      const action = actionMap[btn.dataset.action];
      if (action) action();
    });
  });

  updateDisplay();

```

8. কোথায় বসবে

এই সম্পূর্ণ কোডটি `renderer/renderer.js` ফাইলে বসবে — এটাই এই ফাইলের প্রথম সংস্করণ। পরবর্তী প্রতিটি ফিচারে আমরা এই ফাইলে **নতুন কোড যোগ করব** (প্রতিস্থাপন নয়), এবং `actionMap`-এ নতুন এন্ট্রি যোগ করব।

৫. কেন এই কোড দরকার

- **State ভেরিয়েবল:** `currentInput` (এখন যা টাইপ হচ্ছে বা ফলাফল), `previousInput` (আগের সংখ্যা), `operator` (নির্বাচিত অপারেটর), `resultJustShown` (একটি flag — এইমাত্র = চাপার পর নতুন সংখ্যা টাইপ শুরু হলে পুরনো ফলাফল মুছে নতুন করে শুরু করার জন্য)।
- **`calculate()` :** বিশুদ্ধ (pure) একটি ফাংশন — শুধু ইনপুট নেয়, ফলাফল রিটার্ন করে, কোনো state পরিবর্তন করে না। এটি টেস্ট করা সহজ করে তোলে।
- **0 দিয়ে ভাগ :** `b === 0` চেক করে সরাসরি 'Error' স্ট্রিং রিটার্ন করা হয়, যাতে Infinity না দেখিয়ে সরাসরি Error দেখানো যায়।
- **`inputNumber()` :** ডাবল ডট (`.`) ঠেকানো হয়েছে (`currentInput.includes('.')`), এবং 0 থেকে শুরু করে নতুন সংখ্যা টাইপ করলে অপ্রয়োজনীয় শুরুর শূন্য (leading zero) সরানো হয়েছে।
- **`chooseOperator()` :** যদি ইতিমধ্যে একটি অপারেটর নির্বাচিত থাকে এবং ব্যবহারকারী আরেকটি সংখ্যা টাইপ করার পর আরেকটি অপারেটরে ক্লিক করে (যেমন `5 + 3 +`), তাহলে প্রথমে আগের হিসাব সম্পন্ন করে তারপর নতুন অপারেটর সেট করা হয় — এটি ক্যালকুলেটরের প্রচলিত (chained) আচরণ।

৬. রান করা

```
npm start
```

৭. Testing

নিচের প্রতিটি হিসাব ম্যানুয়ালি টাইপ করে টেস্ট করুন:

ইনপুট	প্রত্যাশিত ফলাফল
<code>2 + 3 =</code>	<code>5</code>
<code>10 - 4 =</code>	<code>6</code>
<code>6 * 7 =</code>	<code>42</code>
<code>20 / 5 =</code>	<code>4</code>
<code>10 / 0 =</code>	<code>Error</code>

এই প্রতিটি ফলাফল Node.js দিয়ে আলাদাভাবে যাচাই করা হয়েছে:

```
node -e "  
function calculate(a, op, b) {
```

```

a = parseFloat(a); b = parseFloat(b);
if (op === '+') return a + b;
if (op === '-') return a - b;
if (op === '*') return a * b;
if (op === '/') return b === 0 ? 'Error' : a / b;
}
console.log(calculate(2,'+',3), calculate(10,'-',4),
calculate(6,'*',7), calculate(20,'/',5), calculate(10,'/',0));
"

```

আউটপুট: 5 6 42 4 Error — সব মিলে গেছে।

৮. প্রত্যাশিত ফলাফল

প্রতিটি হিসাবের পর ডিসপ্লেতে সঠিক ফলাফল দেখাবে, এবং `Error` অবস্থায় নতুন সংখ্যা টাইপ শুরু করলে স্বয়ংক্রিয়ভাবে রিসেট হয়ে নতুন ইনপুট নেবে।

৯. সাধারণ সমস্যা

- **বাটনে ক্লিক করলে কিছু হয় না:** `data-number` / `data-action` attribute বানান ভুল থাকলে অথবা `index.html` -এ `renderer.js` লোড না হলে এমন হয়।
- **NaN দেখা যাচ্ছে:** `previousInput` খালি (`' '`) থাকা অবস্থায় সরাসরি `equals()` চাপলে এটি ঘটতে পারে — কোডে `operator === null` চেক থাকায় এটি প্রতিরোধ করা আছে।

১০. সমাধান

ব্রাউজার DevTools Console (Ctrl+Shift+I) খুলে এরর মেসেজ পড়ুন — সাধারণত ভুল ফাইল/আইডি নামের কারণে “Cannot read property of null” জাতীয় এরর দেখাবে।

অধ্যায় ১২ — Feature ৬: বৈজ্ঞানিক ফাংশন (Scientific Features)

১. Goal

$\sqrt{x}$ (Square Root), x^2 (Square), x^y (Power), $\log$ (Base-10 Logarithm), এবং π (Pi) যোগ করা।

২. জড়িত ফাইল

- `renderer/renderer.js`

৩. Implementation

`renderer.js` ফাইলে নিচের কোডটি যোগ করুন (আগের কোড মুছবেন না):

```
// ===== SCIENTIFIC FUNCTIONS =====
function applyUnary(action) {
  if (currentInput === 'Error') return;
  const value = parseFloat(currentInput);
  let result;

  switch (action) {
    case 'sqrt':
      result = value < 0 ? 'Error' : Math.sqrt(value);
      break;
    case 'square':
      result = value * value;
      break;
    case 'log':
      result = value <= 0 ? 'Error' : Math.log10(value);
      break;
    default:
      return;
  }

  if (result === 'Error') {
    setError();
    return;
  }

  currentInput = result.toString();
}
```

```

    resultJustShown = true;
    updateDisplay();
  }

  function insertPi() {
    currentInput = Math.PI.toString();
    resultJustShown = true;
    updateDisplay();
  }

```

এরপর `actionMap` অবজেক্টে (আগের ধাপে তৈরি করা) নিচের এন্ট্রিগুলো যোগ করুন:

```

sqrt: () => applyUnary('sqrt'),
square: () => applyUnary('square'),
log: () => applyUnary('log'),
pi: insertPi,
power: () => chooseOperator('^')

```

`calculate()` ফাংশনে `^` (power) অপারেটরের সাপোর্ট যোগ করতে `switch` স্টেটমেন্টে নতুন `case` যোগ করুন:

```

case '^': {
  const result = Math.pow(a, b);
  return isFinite(result) ? result : 'Error';
}

```

৪. কোথায় বসবে

- `applyUnary` এবং `insertPi` — `renderer.js`-এ, আগের `calculate / updateDisplay` ফাংশনগুলোর পরে, `actionMap` তৈরি হওয়ার আগে যেকোনো জায়গায়।
- `actionMap`-এর নতুন এন্ট্রি — বিদ্যমান `actionMap` অবজেক্টের ভেতরে।
- `^` কেস — `calculate()` ফাংশনের `switch` ব্লকের ভেতরে, `case '/'` এর পরে।

৫. কেন এই কোড দরকার

- **`applyUnary`**: “unary” মানে যে ফাংশন একটি মাত্র সংখ্যা নিয়ে কাজ করে (`sqrt`, `square`, `log` — এগুলোর জন্য দুটি সংখ্যার দরকার নেই, শুধু বর্তমান

ইনপুট লাগে)। তাই একটি সাধারণ (generic) ফাংশন লেখা হয়েছে, যাতে ভবিষ্যতে নতুন unary ফাংশন যোগ করা সহজ হয়।

- **ঋণাত্মক সংখ্যার Square Root:** গাণিতিকভাবে বাস্তব সংখ্যার মধ্যে ঋণাত্মক সংখ্যার বর্গমূল সংজ্ঞায়িত নয়, তাই `value < 0` হলে `Error` দেখানো হয়।
- **Logarithm-এর ডোমেইন:** `log(0)` এবং ঋণাত্মক সংখ্যার `log` গাণিতিকভাবে অসংজ্ঞায়িত (`undefined`), তাই `value <= 0` হলে `Error`।
- **Power-এর isFinite চেক:** অত্যন্ত বড় সংখ্যার power (যেমন 10^{1000}) জাভাস্ক্রিপ্টে `Infinity` রিটার্ন করে — এটিকেও আমরা `Error` হিসেবে দেখাই যাতে বিভ্রান্তিকর `Infinity` টেক্সট ডিসপ্লেতে না আসে।
- **Math.PI :** জাভাস্ক্রিপ্টের বিল্ট-ইন ধ্রুবক, যা প্রায় ১৫ ঘর দশমিক নির্ভুলতা (`precision`) দেয় — একটি ক্যালকুলেটরের জন্য যথেষ্ট নির্ভুল।

৬. রান করা

```
npm start
```

৭. Testing

ইনপুট	প্রত্যাশিত ফলাফল
<code>sqrt(25)</code>	5
<code>5²</code> (5 লিখে <code>x²</code> চাপুন)	25
<code>2 x^y 8 =</code>	256
<code>log(100)</code>	2
<code>π</code>	3.141592653589793
<code>sqrt(-1)</code>	Error
<code>log(0)</code>	Error
<code>log(-1)</code>	Error

এই ফলাফলগুলো Node.js দিয়ে যাচাই করা হয়েছে:

```
sqrt(25)= 5
2^8= 256
log(100)= 2
log(0)= Error
log(-1)= Error
sqrt(-1)= Error
```

৮. প্রত্যাশিত ফলাফল

প্রতিটি বৈজ্ঞানিক ফাংশন সঠিক সংখ্যাগত ফলাফল দেখাবে, এবং অসংজ্ঞায়িত ইনপুটে (`sqrt(-1)`, `log(0)`, `log(-1)`) `Error` দেখাবে, অ্যাপ ক্র্যাশ করবে না।

৯. সাধারণ সমস্যা

- **`xy` কাজ করছে না:** `calculate()` ফাংশনে `case '^'` যোগ করা হয়নি — আবার যাচাই করুন।
- **`NaN` দেখাচ্ছে বৃহৎ সংখ্যার `power`-এ:** `isFinite()` চেক বাদ পড়ে গেলে এমন হতে পারে।

১০. সমাধান

`switch` ব্লকের প্রতিটি `case`-এর বানান (`'^'`, `'sqrt'`, ইত্যাদি) `data-action` attribute-এর সাথে হুবহু মিলছে কিনা যাচাই করুন — জাভাস্ক্রিপ্টে স্ট্রিং তুলনা case-sensitive।

অধ্যায় ১৩ — Feature ৭: ত্রিকোণমিতি (Trigonometry) + DEG/RAD

১. Goal

`sin`, `cos`, `tan` ফাংশন যোগ করা, ডিগ্রি (DEG) ও রেডিয়ান (RAD) মোডের মধ্যে টগল করার সুবিধাসহ, এবং ফ্লোটিং-পয়েন্ট ভুল (floating-point artifact) সমাধান করা।

২. জড়িত ফাইল

- `renderer/renderer.js`

৩. Implementation

```
// ===== TRIGONOMETRY STATE =====
let angleMode = 'DEG'; // or 'RAD'
const angleModeEl = document.getElementById('angleMode');

function toRadians(value) {
  return angleMode === 'DEG' ? (value * Math.PI) / 180 : value;
}

function cleanFloat(num) {
  return Math.abs(num) < 1e-10 ? 0 : Math.round(num * 1e12) / 1e12;
}

function safeTrig(fn, value) {
  const rad = toRadians(value);
  if (fn === 'tan' && Math.abs(Math.cos(rad)) < 1e-10) return 'Error';
  const result = fn === 'sin' ? Math.sin(rad) : fn === 'cos' ? Math.cos(rad) : Math.tan(rad);
  return cleanFloat(result);
}

function toggleAngleMode() {
  angleMode = angleMode === 'DEG' ? 'RAD' : 'DEG';
}
```

```
updateDisplay();
}
```

`applyUnary` -এর `switch` ব্লকে নতুন কেস যোগ করুন:

```
case 'sin':
  result = safeTrig('sin', value);
  break;
case 'cos':
  result = safeTrig('cos', value);
  break;
case 'tan':
  result = safeTrig('tan', value);
  break;
```

`updateDisplay()` ফাংশনে `angle mode` দেখানোর লাইন যোগ করুন:

```
angleModeEl.textContent = angleMode;
```

`actionMap` -এ যোগ করুন:

```
sin: () => applyUnary('sin'),
cos: () => applyUnary('cos'),
tan: () => applyUnary('tan'),
'deg-rad': toggleAngleMode
```

৪. কোথায় বসবে

`toRadians`, `cleanFloat`, `safeTrig`, `toggleAngleMode` — `renderer.js` -এ `applyUnary` ফাংশনের আগে বা পরে যেকোনো জায়গায় (তবে `applyUnary` -এর ভেতর থেকে `safeTrig` কল হওয়ার আগে ফাংশনটি সংজ্ঞায়িত থাকতে হবে — জাভাস্ক্রিপ্ট ফাংশন ঘোষণা `hoist` হয় বলে সাধারণত এটি সমস্যা করে না)।

৫. কেন এই কোড দরকার

ডিগ্রি বনাম রেডিয়ান: গণিতে কোণ মাপার দুটি একক আছে — ডিগ্রি (০-৩৬০) এবং রেডিয়ান (০-২ π)। জাভাস্ক্রিপ্টের `Math.sin()`, `Math.cos()`, `Math.tan()` ফাংশনগুলো **শুধু রেডিয়ান** ইনপুট নেয়। তাই ব্যবহারকারী যদি DEG মোডে থাকে এবং ৩০ লেখে `sin` চাপে, আমাদের প্রথমে ৩০ ডিগ্রিকে রেডিয়ানে রূপান্তর করতে হবে:

```
radians = degrees × (π / 180)
```

এই কাজটি করে `toRadians()` ফাংশন।

Tangent-এর Asymptote সমস্যা: `tan(90°)` গাণিতিকভাবে অসংজ্ঞায়িত (অসীমের দিকে যায়), কারণ `tan(x) = sin(x) / cos(x)` এবং `cos(90°) = 0` — শূন্য দিয়ে ভাগ। কম্পিউটারে ফ্লোটিং-পয়েন্ট গণনার কারণে `cos(90°)` ঠিক শূন্য না হয়ে একটি অতি ক্ষুদ্র সংখ্যা (যেমন `6.12e-17`) হয়, ফলে `tan(90°)` একটি বিভ্রান্তিকর বিশাল সংখ্যা দেখাতে পারে। আমরা `Math.abs(Math.cos(rad)) < 1e-10` চেক করে এই পরিস্থিতি শনাক্ত করি এবং সরাসরি `Error` দেখাই।

ফ্লোটিং-পয়েন্ট আর্টিফ্যাক্ট (Floating-point Artifact): `sin(180°)` গাণিতিকভাবে ঠিক `0`, কিন্তু কম্পিউটার বাইনারি ফ্লোটিং-পয়েন্ট গণনায় `π`-এর সঠিক মান রাখতে না পারার কারণে ফলাফল আসে `1.2246467991473532e-16` — যা গাণিতিকভাবে শূন্যের প্রায় সমান কিন্তু ঠিক শূন্য নয়। এটি সব প্রোগ্রামিং ভাষার একটি সাধারণ সীমাবদ্ধতা (দশমিক সংখ্যা বাইনারিতে সবসময় নিখুঁতভাবে প্রকাশ করা যায় না)। আমাদের `cleanFloat()` ফাংশন এই সমস্যার সমাধান করে: যদি ফলাফলের পরম মান `1e-10`-এর চেয়ে ছোট হয়, তাহলে ধরে নেওয়া হয় এটি আসলে শূন্যই হওয়ার কথা ছিল, এবং সরাসরি `0` রিটার্ন করা হয়। এছাড়া `Math.round(num * 1e12) / 1e12` দিয়ে ১২ ঘর দশমিকের বেশি অপ্রয়োজনীয় নির্ভুলতা (precision) কেটে ফেলা হয়, যাতে `0.50000000000000001`-এর মতো ফলাফল না দেখিয়ে পরিষ্কার `0.5` দেখানো যায়।

৬. রান করা

```
npm start
```

৭. Testing

ইনপুট (DEG মোডে)	প্রত্যাশিত ফলাফল
<code>sin(30)</code>	<code>0.5</code>
<code>cos(60)</code>	<code>0.5</code>
<code>tan(45)</code>	<code>1</code>
<code>tan(90)</code>	<code>Error</code>
<code>sin(180)</code>	<code>0</code> (এবং <code>1.2246...e-16</code> নয়)

Node.js দিয়ে যাচাইকৃত ফলাফল:

```
sin(30 DEG)= 0.5  
cos(60 DEG)= 0.5  
tan(45 DEG)= 1  
tan(90 DEG)= Error  
sin(180 DEG)= 0
```

RAD মোডেও একই কোণের সমতুল্য রেডিয়ান মান (যেমন $30^\circ = 0.5236 \text{ rad}$) দিয়ে ফলাফল একই আসে কিনা পরীক্ষা করুন।

৮. প্রত্যাশিত ফলাফল

DEG/RAD টগল বাটনে ক্লিক করলে উপরের ছোট লেবেলে “DEG” থেকে “RAD” (বা উল্টো) পরিবর্তিত হবে, এবং ত্রিকোণমিতিক ফাংশনগুলো নির্বাচিত মোড অনুযায়ী সঠিক ফলাফল দেবে।

৯. সাধারণ সমস্যা

- **sin(180) এ 1.22e-16 দেখাচ্ছে:** `cleanFloat()` ফাংশন প্রয়োগ করা হয়নি বা `applyUnary`-তে `safeTrig` কল করার ফলাফল সরাসরি ব্যবহার না করে `Math.sin()` সরাসরি কল করা হয়েছে — কোড আবার মিলিয়ে দেখুন।
- **DEG/RAD টগল করলেও ফলাফল পরিবর্তন হচ্ছে না:** `toRadians()` ফাংশনের ভেতরে `angleMode` ভেরিয়েবলটি সঠিকভাবে চেক হচ্ছে কিনা দেখুন।

১০. সমাধান

কনসোলে `console.log(angleMode)` সাময়িকভাবে যোগ করে টগল বাটন চাপার পর মান পরিবর্তিত হচ্ছে কিনা যাচাই করুন, তারপর লাইনটি সরিয়ে ফেলুন।

অধ্যায় ১৪ — ক্যালকুলেটরের State Management ব্যাখ্যা

আমাদের ক্যালকুলেটর নিচের state variable-গুলো মনে রাখে (সব `renderer.js`-এর শুরুতে `let` দিয়ে ঘোষিত):

ভেরিয়েবল	কাজ
<code>currentInput</code>	এই মুহুর্তে ডিসপ্লেতে যা দেখাচ্ছে (টাইপ করা সংখ্যা বা সর্বশেষ ফলাফল)
<code>previousInput</code>	অপারেটর চাপার আগের সংখ্যাটি
<code>operator</code>	বর্তমানে নির্বাচিত অপারেটর (<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>^</code>) অথবা <code>null</code>
<code>angleMode</code>	<code>'DEG'</code> অথবা <code>'RAD'</code>
<code>resultJustShown</code>	<code>=</code> চাপার পরপরই <code>true</code> হয়ে যায়, যাতে পরের সংখ্যা টাইপে পুরনো ফলাফল মুছে নতুন শুরু হয়
<code>isMuted</code>	সাইলেন্ট বন্ধ কিনা (অধ্যায় ১৯)

State flow: ব্যবহারকারী একটি বাটনে ক্লিক করে → সংশ্লিষ্ট হ্যান্ডলার ফাংশন state ভেরিয়েবলগুলো পরিবর্তন করে → `updateDisplay()` কল হয়ে নতুন state DOM-এ প্রতিফলিত হয়। এই এক-মুখী (one-directional) ডেটা ফ্লো ছোট অ্যাপ্লিকেশনে state ব্যবস্থাপনা সহজ ও অনুমানযোগ্য রাখে — কোনো জায়গায় বাগ হলে সহজেই বোঝা যায় কোন ফাংশনে সমস্যা।

History, Theme, Mute — এই তিনটি state শুধু মেমোরিতে না রেখে `localStorage`-এও সংরক্ষণ করা হয়, যাতে অ্যাপ বন্ধ করে আবার চালু করলেও এগুলো মনে থাকে (পরবর্তী অধ্যায়ে বিস্তারিত)।

অধ্যায় ১৫ — Feature ৮: History System (localStorage)

১. Goal

প্রতিটি হিসাব সম্পন্ন হওয়ার পর তা `localStorage` -এ সংরক্ষণ করা, যাতে অ্যাপ বন্ধ করে আবার চালু করলেও পুরোনো হিসাব মনে থাকে।

২. জড়িত ফাইল

- `renderer/renderer.js`

৩. Implementation

```
// ===== HISTORY (localStorage) =====
function loadHistory() {
  const raw = localStorage.getItem('calc-history');
  return raw ? JSON.parse(raw) : [];
}

function saveHistory(history) {
  localStorage.setItem('calc-history',
    JSON.stringify(history));
}

function addToHistory(expression, result) {
  const history = loadHistory();
  history.unshift({ expression, result });
  if (history.length > 50) history.pop();
  saveHistory(history);
}
```

`equals()` এবং `applyUnary()` ফাংশনের ভেতরে, সফল ফলাফল পাওয়ার পর `addToHistory(...)` কল যোগ করুন। যেমন `equals()` -এ:

```
function equals() {
  if (operator === null || currentInput === 'Error') return;
  const result = calculate(previousInput, operator,
    currentInput);
  const expression = `${previousInput} ${operator} $
```

```
{currentInput}`;

    if (result === 'Error') {
        setError();
        return;
    }

    addToHistory(expression, result.toString()); // নতুন লাইন
    currentInput = result.toString();
    previousInput = '';
    operator = null;
    resultJustShown = true;
    updateDisplay();
}
```

৪. কোথায় বসবে

`loadHistory`, `saveHistory`, `addToHistory` — `renderer.js`-এ, `equals()` ফাংশনের আগে যেকোনো জায়গায়। `addToHistory(...)` কল হবে `equals()` ও `applyUnary()`-এর ভেতরে, ফলাফল সফলভাবে গণনা হওয়ার পর।

৫. কেন এই কোড দরকার

- **localStorage কী:** এটি ব্রাউজার/Electron-এর একটি সাধারণ key-value স্টোরেজ, যা ডিস্কে স্থায়ীভাবে সংরক্ষিত থাকে — অ্যাপ বন্ধ করলেও ডেটা হারায় না (RAM-এর মতো নয়)।
- **কেন এখানে উপযুক্ত:** আমাদের হিস্টোরি ডেটা ছোট (কয়েকশ এন্ট্রি পর্যন্ত), সরল key-value আকারে, এবং জটিল কুয়েরির দরকার নেই — এই ধরনের ব্যবহারের জন্য একটি সম্পূর্ণ ডাটাবেস (যেমন SQLite) অপ্রয়োজনীয় জটিলতা যোগ করত।
- **JSON.stringify():** `localStorage` শুধু স্ট্রিং সংরক্ষণ করতে পারে, অবজেক্ট/অ্যারে নয়। তাই আমাদের হিস্টোরি অ্যারে (যেমন `[{expression, result}, ...]`) কে একটি স্ট্রিং-এ রূপান্তর করতে হয়।
- **JSON.parse():** স্ট্রিং থেকে আবার জাভাস্ক্রিপ্ট অ্যারে/অবজেক্টে ফিরিয়ে আনে, যাতে আমরা `.unshift()`, `.pop()` ইত্যাদি মেথড ব্যবহার করতে পারি।
- **history.length > 50 চেক:** হিস্টোরি অসীমভাবে বাড়তে না দিয়ে সর্বোচ্চ ৫০টি এন্ট্রি রাখা হয়, যাতে `localStorage` অপ্রয়োজনীয়ভাবে বড় না হয়।

৬. রান করা

```
npm start
```

৭. Testing

1. কয়েকটি হিসাব করুন (যেমন $2+2=$, $10*5=$)।
2. অ্যাপ সম্পূর্ণ বন্ধ করুন (উইন্ডো ক্লোজ করুন)।
3. `npm start` দিয়ে আবার চালু করুন।
4. DevTools Console-এ গিয়ে টাইপ করুন: `localStorage.getItem('calc-history')` — সংরক্ষিত হিসাবগুলো JSON আকারে দেখা উচিত।

৮. প্রত্যাশিত ফলাফল

অ্যাপ পুনরায় চালু করার পরও `localStorage.getItem('calc-history')` আগের হিসাবগুলো ফেরত দেবে (যদিও এই ধাপে UI-তে এখনো হিস্টোরি প্যানেল দেখানো হয়নি — পরের অধ্যায়ে তা যোগ হবে)।

৯. সাধারণ সমস্যা

- **localStorage খালি থাকছে:** `addToHistory` কল করা হয়নি কোথাও, অথবা এরর হয়ে ফাংশনটি সম্পন্ন হচ্ছে না — DevTools Console-এ এরর আছে কিনা দেখুন।
- **JSON.parse এরর:** `localStorage` -এ আগে থেকে অবৈধ (corrupt) ডেটা থাকলে হতে পারে। DevTools Console-এ `localStorage.clear()` চালিয়ে রিসেট করুন।

১০. সমাধান

`localStorage.clear()` দিয়ে সব ডেটা মুছে আবার নতুন করে টেস্ট করুন।

অধ্যায় ১৬ — Feature ৯: History Drawer UI

১. Goal

ডান পাশে স্লাইড হয়ে আসা একটি হিস্টোরি প্যানেল তৈরি করা, যেখানে প্রতিটি আগের হিসাব দেখা যাবে, ক্লিক করে পুনরায় ব্যবহার (reuse) করা যাবে, এবং পুরো হিস্টোরি মুছে ফেলা যাবে।

২. জড়িত ফাইল

- `renderer/renderer.js`
- `renderer/styles.css`

৩. Implementation

`renderer.js` -এ যোগ করুন:

```
const historyPanel = document.getElementById('historyPanel');
const historyList = document.getElementById('historyList');

function renderHistory() {
  const history = loadHistory();
  historyList.innerHTML = '';
  history.forEach((item) => {
    const div = document.createElement('div');
    div.className = 'history-item';
    div.innerHTML = `<div class="expr">${item.expression}</div><div class="result">= ${item.result}</div>`;
    div.addEventListener('click', () => {
      currentInput = item.result;
      resultJustShown = true;
      updateDisplay();
    });
    historyList.appendChild(div);
  });
}

document.getElementById('clearHistory').addEventListener('click',
() => {
  saveHistory([]);
  renderHistory();
});
```

```

document.getElementById('historyToggle').addEventListener('click',
() => {
  historyPanel.classList.toggle('open');
});

document.getElementById('closeHistory').addEventListener('click',
() => {
  historyPanel.classList.remove('open');
});

renderHistory();

```

গুরুত্বপূর্ণ: `addToHistory()` ফাংশনের শেষে `renderHistory()`; কল যোগ করুন, যাতে নতুন হিসাব যোগ হওয়ার সাথে সাথে UI আপডেট হয়:

```

function addToHistory(expression, result) {
  const history = loadHistory();
  history.unshift({ expression, result });
  if (history.length > 50) history.pop();
  saveHistory(history);
  renderHistory(); // নতুন লাইন
}

```

`styles.css` -এ যোগ করুন:

```

.history-panel {
  position: fixed;
  top: 0;
  right: -320px;
  width: 300px;
  height: 100%;
  background: var(--app-bg);
  border-left: 1px solid var(--border);
  padding: 16px;
  transition: right 0.25s ease;
  overflow-y: auto;
}

.history-panel.open {
  right: 0;
}

.history-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
}

```

```

margin-bottom: 12px;
font-weight: bold;
}

.history-item {
background: var(--display-bg);
border-radius: 6px;
padding: 8px;
margin-bottom: 8px;
cursor: pointer;
font-size: 13px;
}

.history-item .expr {
opacity: 0.6;
}

.history-item .result {
font-size: 16px;
}

.full-width {
width: 100%;
}

```

৪. কোথায় বসবে

JavaScript অংশ `renderer.js` -এ (History সেকশনের ফাংশনগুলোর কাছাকাছি), CSS অংশ `styles.css` -এর শেষে।

৫. কেন এই কোড দরকার

- **position: fixed + right: -320px**: প্যানেলটি ডিফল্টভাবে স্ক্রিনের বাইরে (ডানদিকে লুকানো) থাকে। `.open` ক্লাস যোগ হলে `right: 0` হয়ে যায়, এবং `transition` থাকায় এটি মসৃণভাবে স্লাইড করে ভেতরে আসে।
- **innerHTML দিয়ে টেমপ্লেট তৈরি**: প্রতিটি হিস্টোরি আইটেমের জন্য আলাদা `<div>` তৈরি করে এতে এক্সপ্রেশন ও ফলাফল বসানো হয়।
- **Click-to-reuse**: প্রতিটি হিস্টোরি আইটেমে ক্লিক ইভেন্ট যোগ করা আছে, যা সেই হিসাবের ফলাফলকে সরাসরি বর্তমান ইনপুট হিসেবে বসিয়ে দেয় — ব্যবহারকারী আগের ফলাফল দিয়ে নতুন হিসাব চালিয়ে যেতে পারে।

- **renderHistory()** বারবার কল করা: প্রতিবার হিস্টোরি পরিবর্তন হলে (নতুন যোগ, বা Clear) সম্পূর্ণ তালিকা আবার আঁকা (re-render) হয় — ছোট তালিকার জন্য এটি সহজ ও যথেষ্ট performant।

৬. রান করা

```
npm start
```

৭. Testing

1. হিস্টোরি আইকনে ক্লিক করুন — প্যানেল ডান থেকে স্লাইড করে আসা উচিত।
2. কিছু হিসাব করে প্যানেল খুলুন — নতুন এন্ট্রি সবার উপরে দেখা উচিত।
3. একটি হিস্টোরি আইটেমে ক্লিক করুন — সেই ফলাফল ডিসপ্লেতে বসে যাওয়া উচিত।
4. “Clear History” চাপুন — তালিকা খালি হয়ে যাওয়া উচিত।
5. বন্ধ বাটন (X) চাপুন — প্যানেল আবার ডানদিকে লুকিয়ে যাওয়া উচিত।

৮. প্রত্যাশিত ফলাফল

সব কয়টি ইন্টারঅ্যাকশন মসৃণভাবে (স্লাইড অ্যানিমেশনসহ) কাজ করবে।

৯. সাধারণ সমস্যা

- প্যানেল স্লাইড না করে হুট করে দেখা যাচ্ছে: `transition: right 0.25s ease` লাইনটি বাদ পড়েছে কিনা দেখুন।
- ক্লিক করলে হিস্টোরি রিলোড হচ্ছে না: `addToHistory()` -এর শেষে `renderHistory()` কল যোগ করা হয়েছে কিনা যাচাই করুন।

১০. সমাধান

CSS ফাইল সেভ করার পর অ্যাপ রিস্টার্ট করুন (Electron স্বয়ংক্রিয়ভাবে CSS রিলোড করে না)।

অধ্যায় ১৭ — Feature ১০: ক্লিপবোর্ড সিস্টেম

১. Goal

ডিসপ্লের ফলাফল কপি করা (বাটনে ক্লিক করে অথবা `Ctrl+C` / `Cmd+C` দিয়ে), এবং ক্লিপবোর্ড থেকে সংখ্যা পেস্ট করা (`Ctrl+V` / `Cmd+V`), সঠিকভাবে স্যানিটাইজ (sanitize) করে।

২. জড়িত ফাইল

- `renderer/renderer.js` (`preload.js` আগেই তৈরি করা হয়েছে অধ্যায় ৮-এ)

৩. Implementation

```
// ===== CLIPBOARD =====
async function copyResult() {
  await window.electronAPI.copyToClipboard(currentInput);
  flashCopyFeedback();
}

function flashCopyFeedback() {
  const original = currentOperandEl.style.color;
  currentOperandEl.style.color = '#4caf7d';
  setTimeout(() => {
    currentOperandEl.style.color = original;
  }, 200);
}

function sanitizeNumeric(text) {
  const cleaned = text.replace(/,/g, '').trim();
  return /^-?\d*\.\?\d+$/.test(cleaned) ? cleaned : null;
}

async function pasteValue() {
  const text = await window.electronAPI.readFromClipboard();
  const sanitized = sanitizeNumeric(text);
  if (sanitized === null) return;
  currentInput = sanitized;
  resultJustShown = false;
  updateDisplay();
}
```

actionMap -এ যোগ করুন:

```
copy: copyResult
```

কীবোর্ড শর্টকাটের জন্য `keydown` লিসেনারে (অধ্যায় ২০-এ পূর্ণাঙ্গভাবে যোগ করা হবে) যোগ করুন:

```
if ((e.ctrlKey || e.metaKey) && e.key.toLowerCase() === 'c') {  
  copyResult();  
}  
if ((e.ctrlKey || e.metaKey) && e.key.toLowerCase() === 'v') {  
  pasteValue();  
}
```

৪. কোথায় বসবে

`copyResult`, `flashCopyFeedback`, `sanitizeNumeric`, `pasteValue` — `renderer.js` -এ। `actionMap` -এ `copy` এন্ট্রি। কীবোর্ড অংশ অধ্যায় ২০-এর `keydown` ইভেন্ট লিসেনারের ভেতরে একত্রিত হবে।

৫. কেন এই কোড দরকার

- **window.electronAPI**: এটি সেই ব্রিজ যা আমরা `preload.js` -এ `contextBridge` দিয়ে এক্সপোজ করেছিলাম (অধ্যায় ৮)। `Renderer` সরাসরি `Electron`-এর `clipboard` মডিউল স্পর্শ করে না — শুধু এই নির্দিষ্ট, সীমিত ফাংশন দুটি ব্যবহার করে।
- **Paste sanitization প্রয়োজন কেন**: ব্যবহারকারী যদি ক্যালকুলেটরের বাইরে থেকে (যেমন কোনো এক্সেল শিট বা ওয়েবসাইট থেকে) "1,234.56" কপি করে পেস্ট করে, তাহলে কমা (,) এবং স্পেস থাকতে পারে যা সরাসরি সংখ্যা হিসেবে ব্যবহারযোগ্য নয়। `sanitizeNumeric()` প্রথমে কমা সরিয়ে, তারপর `regex (/^-?\d*\.\d+$/)` দিয়ে যাচাই করে যে ফলাফলটি আসলেই একটি বৈধ সংখ্যা কিনা। বৈধ না হলে `null` রিটার্ন করে এবং পেস্ট বাতিল হয় — এতে ক্যালকুলেটরে অবৈধ টেক্সট ঢুকে অ্যাপ ভেঙে যাওয়া প্রতিরোধ হয়।
- **ভিজ্যুয়াল ফিডব্যাক**: `flashCopyFeedback()` সংক্ষিপ্ত সময়ের জন্য ডিসপ্লের টেক্সট রঙ সবুজ করে দেয়, যাতে ব্যবহারকারী বুঝতে পারে কপি সফল হয়েছে (কোনো `alert` বা `popup` ছাড়াই, যা ব্যবহারকারীর প্রবাহে বিঘ্ন ঘটায় না)।
- **নিরাপত্তা প্রসঙ্গ**: ক্লিপবোর্ড থেকে আসা যেকোনো টেক্সটকে "অবিশ্বস্ত" (untrusted) ধরে নেওয়া উচিত এবং ব্যবহারের আগে যাচাই করা

উচিত — আমাদের অ্যাপে যেহেতু এই টেক্সট সরাসরি কোনো কোড execute করে না, ব্লক কম, তবু ভুল ইনপুট থেকে UI ভেঙে যাওয়া এড়াতে যাচাই জরুরি।

৬. রান করা

```
npm start
```

৭. Testing

1. একটি হিসাব করুন, ফলাফল আসার পর “copy” বাটনে ক্লিক করুন — ডিসপ্লের টেক্সট সংক্ষিপ্তভাবে সবুজ হওয়া উচিত।
2. একটি টেক্সট এডিটরে গিয়ে `Ctrl+V` / `Cmd+V` চাপুন — কপি করা সংখ্যাটি পেস্ট হওয়া উচিত।
3. কোনো টেক্সট এডিটরে `1,234.56` লিখে কপি করুন, তারপর ক্যালকুলেটরে ফিরে `Ctrl+V` চাপুন — ডিসপ্লেরে `1234.56` (কমা ছাড়া) বসা উচিত।
4. `hello` টাইপ করে কপি করে ক্যালকুলেটরে পেস্ট করার চেষ্টা করুন — কিছুই পরিবর্তন হওয়া উচিত নয় (অবৈধ ইনপুট প্রত্যাখ্যাত)।

৮. প্রত্যাশিত ফলাফল

বৈধ সংখ্যা সঠিকভাবে পেস্ট হবে, অবৈধ টেক্সট প্রত্যাখ্যাত হবে, এবং কপি করলে সিস্টেম ক্লিপবোর্ডে প্রকৃতপক্ষে মান জমা হবে (অন্য অ্যাপে পেস্ট করে যাচাইযোগ্য)।

৯. সাধারণ সমস্যা

- `window.electronAPI is undefined: preload.js` ঠিকভাবে লোড হচ্ছে না — `main.js`-এ পাথ যাচাই করুন।
- **পেস্ট কিছুই করছে না এমনকি বৈধ সংখ্যায়ও:** `regex`-টি সঠিকভাবে টেস্ট হচ্ছে কিনা `console.log(sanitizeNumeric(text))` দিয়ে যাচাই করুন।

১০. সমাধান

DevTools Console-এ ম্যানুয়ালি

```
window.electronAPI.readFromClipboard().then(console.log)
```

 চালিয়ে দেখুন প্রকৃত ক্লিপবোর্ড কন্টেন্ট কী আসছে।

অধ্যায় ১৮ — Feature ১১: Dark/Light থিম

১. Goal

গাঢ় ও হালকা — দুটি থিমের মধ্যে টগল করা, CSS custom properties ব্যবহার করে, এবং নির্বাচিত থিম `localStorage`-এ সংরক্ষণ করা যাতে পরবর্তী সেশনেও মনে থাকে।

২. জড়িত ফাইল

- `renderer/renderer.js`
- `renderer/styles.css`

৩. Implementation

`styles.css`-এ (Stage ১-এ তৈরি `:root` ব্লকের পরে) যোগ করুন:

```
body.light {
  --bg: #eef0f5;
  --app-bg: #ffffff;
  --display-bg: #f4f5f9;
  --text: #202124;
  --btn-number-bg: #e8e9f0;
  --btn-function-bg: #d8dae5;
  --btn-operator-bg: #f0883e;
  --btn-equals-bg: #4caf7d;
  --btn-active: #c7c9d8;
  --border: #d0d2dd;
}
```

`renderer.js`-এ যোগ করুন:

```
// ===== THEME =====
function loadTheme() {
  const saved = localStorage.getItem('calc-theme') || 'dark';
  document.body.classList.toggle('light', saved === 'light');
  document.getElementById('themeToggle').textContent = saved
  === 'light' ? '☀️' : '🌙';
}

function toggleTheme() {
```

```
const isLight = document.body.classList.toggle('light');
localStorage.setItem('calc-theme', isLight ? 'light' :
'dark');
document.getElementById('themeToggle').textContent =
isLight ? '☀️' : '🌙';
}

document.getElementById('themeToggle').addEventListener('click',
toggleTheme);

loadTheme();
```

৪. কোথায় বসবে

CSS অংশ `styles.css` -এর `:root` ব্লকের ঠিক পরে। JavaScript অংশ `renderer.js` -এর যেকোনো জায়গায়, তবে `loadTheme()`; কলটি ফাইলের একদম শেষে (অন্য সব ফাংশন সংজ্ঞায়িত হওয়ার পরে) রাখা ভালো।

৫. কেন এই কোড দরকার

- **CSS Custom Properties (`var(...)`)-এর শক্তি:** যেহেতু আমাদের সব রঙ `:root` -এ ভেরিয়েবল আকারে সংজ্ঞায়িত (যেমন `--bg`, `--text`), তাই থিম পরিবর্তন করতে আমাদের প্রতিটি এলিমেন্টের CSS আলাদাভাবে পরিবর্তন করতে হয় না — শুধু `body` -তে `.light` ক্লাস যোগ/অপসারণ করলেই সব ভেরিয়েবলের নতুন মান কার্যকর হয়ে যায় ব্রাউজার নিজে থেকেই।
- **`classList.toggle('light')` :** এই মেথডটি ক্লাস থাকলে সরিয়ে দেয়, না থাকলে যোগ করে, এবং বর্তমানে ক্লাসটি আছে কিনা তা বুলিয়ান হিসেবে রিটার্ন করে — আমরা সেই রিটার্ন মানই `localStorage` -এ সংরক্ষণ করি।
- **`loadTheme()` চালু হওয়ার সময় কল করা:** অ্যাপ চালু হওয়ার সাথে সাথেই আগের সংরক্ষিত থিম প্রয়োগ করা হয়, যাতে ব্যবহারকারী সবসময় ডার্ক মোডে অ্যাপ শুরু হতে দেখে বিরক্ত না হন যদি তিনি লাইট মোড পছন্দ করে থাকেন।

৬. রান করা

```
npm start
```

৭. Testing

1. থিম টগল বাটনে ক্লিক করুন — পুরো অ্যাপের রঙ dark থেকে light (বা উল্টো) পরিবর্তিত হওয়া উচিত, আইকনও 🌙 থেকে ☀️ পরিবর্তিত হবে।

2. Light মোডে থাকা অবস্থায় অ্যাপ বন্ধ করুন।
3. আবার `npm start` চালান — অ্যাপ light মোডেই চালু হওয়া উচিত।
4. একই টেস্ট dark মোডের জন্যও করুন।

৮. প্রত্যাশিত ফলাফল

থিম পরিবর্তন `transition` অ্যানিমেশনসহ মসৃণভাবে ঘটবে, এবং অ্যাপ পুনরায় চালু করলেও সর্বশেষ নির্বাচিত থিম মনে থাকবে।

৯. সাধারণ সমস্যা

- থিম পরিবর্তন হচ্ছে কিন্তু রিস্টার্টের পর মনে থাকছে না: `loadTheme()` ফাংশন কল করা হয়নি ফাইলের শেষে, অথবা `localStorage.setItem` ভুল key নামে সেভ হচ্ছে।
- কিছু এলিমেন্টের রঙ পরিবর্তন হচ্ছে না: সেই এলিমেন্টের CSS-এ hardcoded রঙ ব্যবহার হয়েছে, `var(--...)` ব্যবহার করা হয়নি — CSS ফাইলে খুঁজে বের করে ভেরিয়েবলে পরিবর্তন করুন।

১০. সমাধান

DevTools-এর Elements ট্যাবে গিয়ে `<body>` এলিমেন্টে `light` ক্লাস যোগ/বাদ হচ্ছে কিনা ম্যানুয়ালি পরীক্ষা করুন।

অধ্যায় ১৯ — Feature ১২: Web Audio API দিয়ে সাউন্ড

১. Goal

কোনো external অডিও ফাইল ছাড়াই, প্রোগ্রাম্যাটিকভাবে (synthesized) সংখ্যা/অপারেটর/ফলাফলের জন্য আলাদা সাউন্ড তৈরি করা, সাথে Mute/Unmute সুবিধা।

২. জড়িত ফাইল

- `renderer/renderer.js`

৩. Implementation

```
// ===== WEB AUDIO =====
let isMuted = false;
let audioCtx = null;

function getAudioContext() {
  if (!audioCtx) {
    audioCtx = new (window.AudioContext ||
window.webkitAudioContext)();
  }
  return audioCtx;
}

function playSound(kind) {
  if (isMuted) return;
  const ctx = getAudioContext();
  const oscillator = ctx.createOscillator();
  const gain = ctx.createGain();

  const frequencies = { number: 440, operator: 550, result:
660, error: 220 };
  oscillator.frequency.value = frequencies[kind] || 440;
  oscillator.type = 'sine';

  gain.gain.setValueAtTime(0.08, ctx.currentTime);
  gain.gain.exponentialRampToValueAtTime(0.0001,
ctx.currentTime + 0.12);

  oscillator.connect(gain);
```

```

gain.connect(ctx.destination);

oscillator.start();
oscillator.stop(ctx.currentTime + 0.12);
}

function loadMuteState() {
  isMuted = localStorage.getItem('calc-muted') === 'true';
  document.getElementById('muteToggle').textContent = isMuted ?
  '🔇' : '🔊';
}

function toggleMute() {
  isMuted = !isMuted;
  localStorage.setItem('calc-muted', isMuted.toString());
  document.getElementById('muteToggle').textContent = isMuted ?
  '🔇' : '🔊';
}

document.getElementById('muteToggle').addEventListener('click',
toggleMute);
loadMuteState();

```

এরপর যথাস্থানে `playSound(...)` কল যোগ করুন:

- `inputNumber()` -এর শেষে: `playSound('number');`
- `chooseOperator()` -এর শেষে: `playSound('operator');`
- `equals()` -এ সফল ফলাফলের পর: `playSound('result');`, এবং error হলে: `playSound('error');`

৪. কোথায় বসবে

Web Audio ব্লক `renderer.js` -এর যেকোনো জায়গায়। `playSound()` কলগুলো সংশ্লিষ্ট ফাংশনগুলোর ভেতরে, state পরিবর্তনের পরে ও `updateDisplay()` -এর আগে বা পরে (ক্রম গুরুত্বপূর্ণ নয়)।

৫. কেন এই কোড দরকার

- **AudioContext** : ব্রাউজার/Electron-এর Web Audio API-এর মূল প্রবেশদ্বার। এর মাধ্যমে আমরা সরাসরি সাউন্ড ওয়েভ তৈরি করতে পারি — কোনো `.mp3` / `.wav` ফাইল লোড না করেই।

- **Oscillator** : একটি নির্দিষ্ট ফ্রিকোয়েন্সির (frequency) বিশুদ্ধ টোন তৈরি করে। আমরা প্রতিটি ইভেন্টের (সংখ্যা, অপারেটর, ফলাফল, এরর) জন্য ভিন্ন ফ্রিকোয়েন্সি ব্যবহার করেছি, যাতে ব্যবহারকারী শব্দ শুনেই বুঝতে পারে কী ঘটেছে।
- **Gain** : সাউন্ডের ভলিউম নিয়ন্ত্রণ করে। আমরা `exponentialRampToValueAtTime` ব্যবহার করেছি যাতে সাউন্ডটি হঠাৎ না থেমে ধীরে ধীরে মিলিয়ে যায় (fade out) — এতে শব্দটি কানে আরামদায়ক শোনায়, খটখটে (clicky) মনে হয় না।
- **কেন generated audio উপযোগী এখানে:** external অডিও ফাইল ব্যবহার করলে প্রজেক্টে বাড়তি ফাইল যোগ করতে হতো, প্যাকেজিং-এর সময় সেগুলো বান্ডিল করতে হতো, এবং লাইসেন্সিং প্রশ্ন উঠতে পারত। Web Audio দিয়ে সাউন্ড তৈরি করলে কোনো ফাইল ছাড়াই, মাত্র কয়েক লাইন কোডে কাজ চলে যায়, এবং ফাইল সাইজ ছোট থাকে।
- **Persistent Mute State:** `localStorage`-এ mute অবস্থা রাখা হয়, যাতে ব্যবহারকারী একবার mute করলে পরের বার অ্যাপ চালু করার সময়ও তা মনে থাকে।

৬. রান করা

```
npm start
```

৭. Testing

1. সংখ্যায় ক্লিক করুন — একটি সংক্ষিপ্ত টোন শোনা উচিত।
2. অপারেটরে ক্লিক করুন — ভিন্ন (কিছুটা উচ্চতর) টোন শোনা উচিত।
3. `=` চাপুন — ফলাফলের টোন শোনা উচিত।
4. `10 / 0 =` করুন — এরর টোন (নিচু স্বর) শোনা উচিত।
5. Mute বাটনে ক্লিক করে আবার কিছু চাপুন — কোনো শব্দ হওয়া উচিত নয়।
6. Mute অবস্থায় অ্যাপ রিস্টার্ট করুন — এখনো mute থাকা উচিত।

৮. প্রত্যাশিত ফলাফল

প্রতিটি ইভেন্টে আলাদা শব্দ শোনা যাবে, Mute করলে সব শব্দ বন্ধ হয়ে যাবে, এবং এই অবস্থা সংরক্ষিত থাকবে।

৯. সাধারণ সমস্যা

- **কোনো শব্দ শোনা যাচ্ছে না:** কিছু অপারেটিং সিস্টেমে `AudioContext` ব্যবহারকারীর প্রথম ইন্টারঅ্যাকশনের (click) আগ পর্যন্ত “suspended” অবস্থায় থাকে — এটি স্বাভাবিক ব্রাউজার নিরাপত্তা নীতি, এবং একবার বাটনে ক্লিক করলেই সমাধান হয়ে যায়।
- **শব্দ শুরুতেই কাটকাট শোনাচ্ছে:** `gain.gain` র‍্যাম্প-আউট সময় (0.12 সেকেন্ড) খুব কম হলে এমন লাগতে পারে — মান কিছুটা বাড়িয়ে দেখুন।

১০. সমাধান

`getAudioContext()` -এ `audioCtx.resume()` কল যোগ করে দেখতে পারেন, যদি কিছু প্ল্যাটফর্মে সমস্যা থেকে যায়।

অধ্যায় ২০ — Feature ১৩: কীবোর্ড সাপোর্ট

১. Goal

মাউস ছাড়াই সম্পূর্ণ কীবোর্ড দিয়ে ক্যালকুলেটর চালানো, প্রতিটি কী চাপার সাথে সংশ্লিষ্ট বাটনের ভিজুয়াল “চাপা” (pressed) ইফেক্ট দেখানো।

২. জড়িত ফাইল

- `renderer/renderer.js`

৩. Implementation

```
// ===== KEYBOARD SUPPORT =====
const keyToSelector = {
  '+': '[data-action="add"]',
  '-': '[data-action="subtract"]',
  '*': '[data-action="multiply"]',
  '/': '[data-action="divide"]',
  '^': '[data-action="power"]',
  Enter: '[data-action="equals"]',
  '=': '[data-action="equals"]',
  Escape: '[data-action="clear"]',
  s: '[data-action="sin"]',
  c: '[data-action="cos"]',
  t: '[data-action="tan"]',
  r: '[data-action="sqrt"]',
  q: '[data-action="square"]',
  p: '[data-action="pi"]',
  l: '[data-action="log"]',
  d: '[data-action="deg-rad"]'
};

function pressVisual(selector) {
  const el = document.querySelector(selector);
  if (el) el.classList.add('pressed');
}

function releaseVisual(selector) {
  const el = document.querySelector(selector);
  if (el) el.classList.remove('pressed');
}
```

```

document.addEventListener('keydown', (e) => {
  if (/^[0-9]$/.test(e.key) || e.key === '.') {
    inputNumber(e.key);
    return;
  }

  if (e.key === 'h') {
    historyPanel.classList.toggle('open');
    return;
  }

  const selector = keyToSelector[e.key];
  if (selector) {
    pressVisual(selector);
    const el = document.querySelector(selector);
    if (el) el.click();
  }

  if ((e.ctrlKey || e.metaKey) && e.key.toLowerCase() === 'c')
  {
    copyResult();
  }
  if ((e.ctrlKey || e.metaKey) && e.key.toLowerCase() === 'v')
  {
    pasteValue();
  }
});

document.addEventListener('keyup', (e) => {
  const selector = keyToSelector[e.key];
  if (selector) releaseVisual(selector);
});

```

৪. কোথায় বসবে

`renderer.js` ফাইলের একদম শেষের দিকে, `loadTheme()`, `loadMuteState()`, `renderHistory()`, `updateDisplay()` কলগুলোর আগে।

৫. কেন এই কোড দরকার

- **keyToSelector** ম্যাপিং: প্রতিটি কীবোর্ড কী-কে সংশ্লিষ্ট বাটনের CSS সিলেক্টরের সাথে ম্যাপ করা হয়েছে। এতে নতুন শর্টকাট যোগ করা সহজ — শুধু এই অবজেক্টে একটি এন্ট্রি যোগ করলেই হয়।
- **el.click()** ব্যবহার: কীবোর্ড ইভেন্টে সরাসরি লজিক না লিখে, আমরা সংশ্লিষ্ট বাটনের `click()` মেথড কল করি। এতে মাউস-ক্লিক এবং কীবোর্ড — দুই ধরনের

ইনপুটই একই কোড পাথ ব্যবহার করে, ফলে লজিক দুইবার লেখার দরকার হয় না (DRY নীতি — Don't Repeat Yourself)।

- **keydown বনাম keyup**: `keydown` কী চাপার মুহুর্তে ঘটে (এখানে আমরা অ্যাকশন সম্পন্ন করি ও `visual "pressed"` ক্লাস যোগ করি), আর `keyup` কী ছাড়ার মুহুর্তে ঘটে (এখানে আমরা `"pressed"` ক্লাস সরিয়ে ফেলি) — এভাবে বাটন ঠিক তত সময় "চাপা" দেখাবে যতক্ষণ কীবোর্ড কী চেপে ধরে রাখা হয়, বাস্তব বাটনের মতো।
- **Ctrl / Cmd উভয় সাপোর্ট**: `e.ctrlKey || e.metaKey` চেক দিয়ে Windows/Linux-এর `Ctrl` এবং macOS-এর `Cmd` — দুটোই একই শর্টকাট হিসেবে কাজ করে, যাতে সব প্ল্যাটফর্মে সমান অভিজ্ঞতা পাওয়া যায়।

৬. রান করা

```
npm start
```

৭. Testing

প্রতিটি শর্টকাট চেপে যাচাই করুন:

কী	প্রত্যাশিত কাজ
0 - 9 , .	সংখ্যা ইনপুট
+ - * /	অপারেটর নির্বাচন
^	Power
Enter / =	সমান
Escape	Clear
s / c / t	sin / cos / tan
r	Square Root
q	Square
p	Pi
l	Log
d	DEG/RAD টগল
h	History প্যানেল টগল
Ctrl+C	কপি
Ctrl+V	পেস্ট

প্রতিটি কী চাপার সময় সংশ্লিষ্ট বাটন সংক্ষিপ্তভাবে “চাপা” রঙে পরিবর্তিত হচ্ছে কিনা লক্ষ্য করুন।

৮. প্রত্যাশিত ফলাফল

মাউস ব্যবহার না করেই সম্পূর্ণ ক্যালকুলেটর কীবোর্ড দিয়ে পরিচালনা করা সম্ভব হবে।

৯. সাধারণ সমস্যা

- **টাইপ করার সময় সংখ্যা দুইবার বসছে:** `keydown` লিসেনারে সংখ্যার জন্য `return` না থাকলে, একই ইভেন্ট পরে আবার `keyToSelector` -এ মিলে গিয়ে দ্বিতীয়বার প্রসেস হতে পারে। কোডে সংখ্যার `if` ব্লকের শেষে `return;` আছে কিনা যাচাই করুন।
- **h চাপলে সংখ্যা হিসেবে কিছু না ঘটেও History না খোলা:**
`keyToSelector` -এ `h` -এর এন্ট্রি না রেখে আলাদা `if (e.key === 'h')` ব্লক ব্যবহার করা হয়েছে (কারণ `historyToggle` বাটনটি `data-action` দিয়ে চিহ্নিত নয়, এটি সরাসরি `id` দিয়ে চিহ্নিত) — কোড ঠিক এই কাঠামো অনুসরণ করছে কিনা দেখুন।

১০. সমাধান

DevTools Console-এ `document.addEventListener('keydown', e => console.log(e.key))` সাময়িকভাবে চালিয়ে দেখুন প্রতিটি কী-প্রেসে সঠিক `e.key` মান আসছে কিনা।

অধ্যায় ২১ — উইন্ডো কনফিগারেশন — সারসংক্ষেপ

অধ্যায় ৭-এ যা সেট করা হয়েছিল তার কারণ সংক্ষেপে মনে করিয়ে দেওয়া হলো:

- `width: 380, height: 680` — একটি মোবাইল ক্যালকুলেটরের মতো সরু, লম্বা অনুপাত, যা সংখ্যার গ্রিড ও ডিসপ্লেজের জন্য স্বাভাবিক দেখায়।
 - `minWidth: 340, minHeight: 560` — এর চেয়ে ছোট করলে বাটনগুলো এত সংকুচিত হয়ে যেত যে চাপা কঠিন হয়ে পড়ত।
 - `title: 'Scientific Calculator'` — টাস্কবার/ডক ও উইন্ডো টাইটেল বারে পরিচিতির জন্য স্পষ্ট নাম।
 - `Menu.setApplicationMenu(null)` — ডিফল্ট File/Edit/View মেনু একটি ক্যালকুলেটরের জন্য অপ্রাসঙ্গিক, তাই লুকানো হয়েছে যাতে UI পরিষ্কার থাকে।
-

অধ্যায় ২২ — এরর হ্যান্ডলিং — সম্পূর্ণ সারসংক্ষেপ

পরিস্থিতি	আচরণ
শূন্য দিয়ে ভাগ ($x / 0$)	ডিসপ্লেতে <code>Error</code> , state রিসেট
ঋণাত্মক সংখ্যার Square Root	<code>Error</code>
<code>Log(0)</code> বা <code>Log(ঋণাত্মক)</code>	<code>Error</code>
Tangent Asymptote ($\tan 90^\circ$)	<code>Error</code> (বিভ্রান্তিকর বিশাল সংখ্যা নয়)
অসীম Power ফলাফল	<code>Error</code> (<code>isFinite</code> চেক দিয়ে)
অবৈধ পেস্ট করা মান	পেস্ট নীরবভাবে প্রত্যাখ্যাত, কোনো পরিবর্তন হয় না
খালি ইনপুট নিয়ে <code>=</code> চাপা	কিছুই ঘটে না (<code>operator === null</code> চেক)
ক্লিপবোর্ড ব্যর্থতা	<code>async/await</code> দ্বারা প্রমিজ হিসেবে হ্যান্ডল হয়; ব্যর্থ হলে অ্যাপ ক্র্যাশ করে না, শুধু কপি/পেস্ট কাজ করে না
<code>localStorage</code> সমস্যা (যেমন কোটা পূর্ণ)	ব্রাউজার এক্সসেপশন থ্রো করবে; ছোট ক্যালকুলেটর হিস্টোরির জন্য এটি বাস্তবে ঘটার সম্ভাবনা নেই, তবে production অ্যাপে <code>try/catch</code> দিয়ে মুড়িয়ে রাখা ভালো চর্চা
অডিও চালু করতে ব্যর্থ হওয়া	<code>AudioContext</code> তৈরি ব্যর্থ হলে <code>playSound()</code> নীরবে কিছু না করেই এগিয়ে যাবে, ক্যালকুলেটরের মূল কাজ ব্যাহত হবে না
Electron স্টার্টআপ এরর	সাধারণত ভুল <code>preload</code> পাথ বা <code>node_modules</code> অনুপস্থিত থাকার কারণে হয় — টার্মিনালে এরর মেসেজ পড়ে সমাধান করতে হবে

মূলনীতি: কোনো একক গাণিতিক বা ইনপুট ভুলের কারণে যেন পুরো অ্যাপ্লিকেশন ক্র্যাশ না করে — বরং নিয়ন্ত্রিতভাবে `Error` দেখিয়ে ব্যবহারকারীকে আবার চেষ্টা করতে দেওয়া হয়।

অধ্যায় ২৩ — সম্পূর্ণ টেস্টিং পরিকল্পনা

নিচের চেকলিস্ট অনুযায়ী পুরো অ্যাপ একবার টেস্ট করুন।

মৌলিক গণিত

☐

$$2 + 2 = 4$$

☐

$$10 - 5 = 5$$

☐

$$4 * 6 = 24$$

☐

$$20 / 4 = 5$$

বৈজ্ঞানিক

☐

$$\text{sqrt}(25) = 5$$

☐

$$5^2 = 25$$

☐

$$2^8 = 256$$

☐

$$\log(100) = 2$$

☐

$$\pi \approx 3.14159\dots$$

ত্রিকোণমিতি (DEG মোডে)

☐

$$\sin(30^\circ) = 0.5$$

☐

$$\cos(60^\circ) = 0.5$$

☐

$$\tan(45^\circ) = 1$$

RAD মোড

☐

একই কোণের রেডিয়ান সমতুল্য মান দিয়ে উপরের ফলাফলগুলো পুনরায় যাচাই করুন।

এজ কেস (Edge Cases)

☐

`1 / 0 → Error`

☐

`sqrt(-1) → Error`

☐

`log(0) → Error`

☐

`log(-1) → Error`

☐

`tan(90°) → Error`

History

☐

হিসাব সেভ হচ্ছে

☐

ক্লিক করলে reuse হচ্ছে

☐

কপি কাজ করছে

☐

Clear কাজ করছে

☐

অ্যাপ রিস্টার্টের পরও হিস্টোরি টিকে থাকছে (persistence)

থিম

☐

Dark মোড

☐

Light মোড

☐

রিস্টার্টের পরও পছন্দ মনে থাকছে

কীবোর্ড

☐

প্রতিটি শর্টকাট (অধ্যায় ২০-এর টেবিল অনুযায়ী) কাজ করছে

ক্লিপবোর্ড

☐

কপি

☐

পেস্ট (বৈধ ও অবৈধ উভয় ইনপুট দিয়ে)

অডিও

☐

সংখ্যার শব্দ

☐

অপারেটরের শব্দ

☐

ফলাফলের শব্দ

☐

Mute

☐

Unmute

অধ্যায় ২৪ — ডিবাগিং ও ট্রাবলশুটিং গাইড

সমস্যা	কারণ	নির্ণয় (Diagnose)	সমাধান
<code>npm: command not found</code>	Node.js ইনস্টল করা নেই বা PATH-এ নেই	<code>node --version</code> চালিয়ে দেখুন	Node.js পুনরায় ইনস্টল করুন, টার্মিনাল রিস্টার্ট করুন
<code>electron: command not found</code>	<code>node_modules</code> ফোল্ডার নেই বা <code>npm install</code> ব্যর্থ হয়েছে	<code>ls node_modules \</code> <code>grep electron</code> (Linux/macOS)	<code>npm install --save-dev electron</code> আবার চালান
অ্যাপ্লিকেশন খুলছে না	<code>main.js</code> -এ syntax এরর	টার্মিনালে এরর মেসেজ পড়ুন	নির্দেশিত লাইন নম্বরে গিয়ে সিনটাক্স ঠিক করুন
খালি (blank) Electron উইন্ডো	<code>loadFile</code> পাথ ভুল, অথবা <code>index.html</code> -এ এরর	DevTools (Ctrl+Shift+I) খুলে Console/Network ট্যাব দেখুন	পাথ ও ফাইলের নাম case-sensitive ভাবে যাচাই করুন
Preload লোড হচ্ছে না	<code>main.js</code> -এ <code>preload</code> পাথ ভুল	DevTools Console-এ <code>window.electronAPI</code> টাইপ করে দেখুন <code>undefined</code> আসছে কিনা	<code>path.join(__dirname, 'preload.js')</code> সঠিক আছে কিনা যাচাই করুন
contextBridge এরর	<code>nodeIntegration: true</code> করা আছে অথচ <code>contextBridge</code> ব্যবহার করার চেষ্টা হচ্ছে, অথবা <code>preload</code> ফাইলে ভুল সিনটাক্স	টার্মিনাল/DevTools এরর লগ পড়ুন	<code>webPreferences</code> সেটিংস অধ্যায় ৭-এর কোডের সাথে মিলিয়ে দেখুন
JavaScript এরর (renderer.js)	টাইপো, ভুল ভেরিয়েবল নাম, <code>undefined</code> ফাংশন কল	DevTools Console-এ লাল রঙের এরর মেসেজ ও লাইন নম্বর পড়ুন	নির্দেশিত লাইনে গিয়ে বানান/সিনটাক্স মিলিয়ে দেখুন
CSS লোড হচ্ছে না	<code><link></code> ট্যাগের <code>href</code> ভুল, অথবা ফাইলের নাম ভুল	DevTools-এর Network ট্যাবে <code>styles.css</code> লাল (404) দেখাচ্ছে কিনা দেখুন	<code>index.html</code> -এ <code>href="styles.css"</code> এবং ফাইলের প্রকৃত নাম/অবস্থান মিলিয়ে দেখুন
বাটন কাজ করছে না	ইভেন্ট লিসেনার যোগ হয়নি, অথবা <code>data-* attribute</code> বানান ভুল	DevTools Console-এ ম্যানুয়ালি <code>document.querySelectorAll('.btn')</code> চালিয়ে বাটনের সংখ্যা মিলিয়ে দেখুন	HTML-এর <code>data-action / data-number</code> এবং JS-এর <code>actionMap</code> কী-গুলো হুবহু মিলিয়ে দেখুন
<code>localStorage</code> কাজ করছে না	Private/Incognito-জাতীয় সীমাবদ্ধতা		<code>key</code> নামের বানান (<code>calc-history</code> , <code>calc-theme</code> ,

সমস্যা	কারণ	নির্ণয় (Diagnose)	সমাধান
	(Electron-এ সাধারণত হয় না) অথবা ভুল key নাম	DevTools Console-এ <code>localStorage.setItem('test', '1')</code> চালিয়ে দেখুন এরর আসে কিনা	<code>calc-muted</code>) মিলিয়ে দেখুন
ক্লিপবোর্ড কাজ করছে না	preload-এ <code>clipboard</code> এক্সপোজ করা হয়নি, বা <code>renderer</code> -এ <code>window.electronAPI</code> ব্যবহার হচ্ছে না	DevTools Console-এ <code>window.electronAPI</code> চেক করুন	preload.js-এর কোড অধ্যায় ৮-এর সাথে মিলিয়ে দেখুন
কীবোর্ড শর্টকাট কাজ করছে না	<code>keydown</code> লিসেনার যোগ হয়নি, অথবা সিলেক্টর ভুল	Console-এ <code>console.log(e.key)</code> দিয়ে যাচাই করুন	<code>keyToSelector</code> ম্যাপিং যাচাই করুন
অডিও শোনা যাচ্ছে না	<code>AudioContext</code> <code>suspended</code> অবস্থায় আছে (প্রথম ইন্টারঅ্যাকশনের আগে)	একবার যেকোনো বাটনে ক্লিক করে আবার চেষ্টা করুন	স্বাভাবিক আচরণ — ব্যবহারকারীর প্রথম ক্লিকের পর অডিও চালু হয়ে যায়
অ্যাপ্লিকেশন ক্র্যাশ করছে	কোনো ধরা-পড়েনি এমন এক্সেপশন	টার্মিনাল থেকে <code>npm start</code> চালিয়ে সম্পূর্ণ স্ট্যাক ট্রেস পড়ুন	ত্রুটিপূর্ণ ফাংশনের চারপাশে <code>try/catch</code> যোগ করুন এবং ইনপুট ভ্যালিডেশন যাচাই করুন

সাধারণ ডিবাগিং কৌশল: সবসময় `Ctrl+Shift+I` (Windows/Linux) বা `Cmd+Option+I` (macOS) দিয়ে DevTools খুলুন — এটি Electron-এ ওয়েব ব্রাউজারের মতোই কাজ করে এবং প্রায় সব JavaScript/CSS সমস্যা এখান থেকেই ধরা যায়।

অধ্যায় ২৫ — নিরাপত্তা (Security) নোট

বিষয়	ব্যাখ্যা
contextIsolation	Renderer ও preload-এর জাভাস্ক্রিপ্ট জগত আলাদা রাখে, যাতে renderer সরাসরি preload-এর অভ্যন্তরীণ ভেরিয়েবল/মডিউলে হাত দিতে না পারে।
nodeIntegration	বন্ধ রাখলে renderer-এর ভেতরের কোনো কোড সরাসরি <code>require()</code> ব্যবহার করে Node.js API-তে প্রবেশ করতে পারে না।
preload	একমাত্র নিয়ন্ত্রিত প্রবেশপথ — শুধু যা explicit ভাবে <code>contextBridge</code> দিয়ে এক্সপোজ করা হয়, তাই ব্যবহারযোগ্য।
contextBridge	preload থেকে renderer-এ নির্দিষ্ট, সীমিত ফাংশন পাঠানোর নিরাপদ পদ্ধতি।
IPC (Inter-Process Communication)	আমাদের ক্যালকুলেটরে সরাসরি IPC চ্যানেল ব্যবহার করা হয়নি (ক্লিপবোর্ড preload-এর মধ্যেই হ্যান্ডল হয়), তবে বড় অ্যাপে <code>ipcMain</code> / <code>ipcRenderer</code> দিয়ে Main ও Renderer-এর মধ্যে বার্তা আদান-প্রদান করা হয়।
XSS বিবেচনা	আমাদের ক্যালকুলেটরে বাইরের কোনো ওয়েব কন্টেন্ট লোড হয় না, তাই ক্লাসিক XSS ঝুঁকি কম। তবু history আইটেম রেন্ডার করার সময় ব্যবহারকারীর কাঁচা ইনপুট সরাসরি <code>innerHTML</code> -এ বসানো এড়িয়ে চলা সাধারণভাবে ভালো চর্চা।
unsafe eval	ক্যালকুলেটরের হিসাব <code>eval()</code> দিয়ে না করে, নিয়ন্ত্রিত <code>switch</code> -ভিত্তিক <code>calculate()</code> ফাংশন দিয়ে করা হয়েছে — এতে ইনপুট হিসেবে সরাসরি জাভাস্ক্রিপ্ট কোড চালানোর ঝুঁকি থাকে না।
input validation	সব সংখ্যাগত ইনপুট (বিশেষত পোস্ট করা মান) <code>sanitizeNumeric()</code> দিয়ে যাচাই করা হয়।
clipboard input	ক্লিপবোর্ড থেকে আসা যেকোনো টেক্সটকে অবিশ্বস্ত ধরে নিয়ে ব্যবহারের আগে regex দিয়ে বৈধতা যাচাই করা হয়।
external URLs	আমাদের অ্যাপ কোনো বাহ্যিক URL লোড করে না (<code>loadFile</code> দিয়ে শুধু লোকাল ফাইল লোড

বিষয়	ব্যাখ্যা
	হয়) — ফলে দূরবর্তী কোডের সাথে যুক্ত হওয়ার ঝুঁকি নেই।

DO

- `contextIsolation: true` এবং `nodeIntegration: false` সবসময় রাখুন।
- preload-এ শুধু প্রয়োজনীয়, নির্দিষ্ট ফাংশন এক্সপোজ করুন।
- সব ব্যবহারকারী ইনপুট (বিশেষত ক্লিপবোর্ড থেকে) যাচাই করুন।
- সংখ্যাগত হিসাব নিয়ন্ত্রিত লজিক দিয়ে করুন, `eval()` এড়িয়ে চলুন।

DON'T

- `nodeIntegration: true` অপ্রয়োজনে চালু করবেন না।
 - পুরো `require('electron')` বা `require('fs')` মডিউল সরাসরি `contextBridge`-এর মাধ্যমে এক্সপোজ করবেন না।
 - ব্যবহারকারীর ইনপুট (বিশেষত পেস্ট করা টেক্সট) `eval()` দিয়ে প্রসেস করবেন না।
 - ক্লিপবোর্ড থেকে আসা মান অন্ধভাবে বিশ্বাস করে সরাসরি ব্যবহার করবেন না।
-

অধ্যায় ২৬ — Do's এবং Don'ts (সাধারণ কোডিং অভ্যাস)

DO

- প্রতিটি ফিচার যোগ করার পরপরই টেস্ট করুন — শেষ পর্যন্ত অপেক্ষা করবেন না।
- ফাংশনগুলোকে ছোট ও একক-দায়িত্বপূর্ণ (single-responsibility) রাখুন।
- অর্থবহ ভেরিয়েবল ও ফাংশনের নাম ব্যবহার করুন (`currentInput` , `applyUnary` — নাম দেখেই কাজ বোঝা যায়)।
- State-কে predictable রাখুন — একটিমাত্র জায়গা থেকে state পরিবর্তন করুন, ছড়িয়ে-ছিটিয়ে নয়।
- এজ কেস (division by zero, negative sqrt ইত্যাদি) সচেতনভাবে টেস্ট করুন।

DON'T

- অপ্রয়োজনে `nodeIntegration` চালু করবেন না।
 - সমস্ত লজিক একটি বিশাল ফাংশনে গুঁজে দেবেন না — ভেঙে ভেঙে লিখুন।
 - ফ্লোটিং-পয়েন্ট গণনার ত্রুটি (artifact) উপেক্ষা করবেন না — `sin(180°)` -এর মতো ক্ষেত্রে এটি ব্যবহারকারীর কাছে বাগ মনে হবে।
 - ক্লিপবোর্ড ইনপুট অন্ধভাবে বিশ্বাস করবেন না।
 - না বুঝে কোড কপি-পেস্ট করবেন না — প্রতিটি অংশ কোথায় বসছে ও কেন, তা বুঝে তারপর বসান।
-

অধ্যায় ২৭ — ডেভেলপমেন্ট ওয়ার্কফ্লো (Git সহ)

সুপারিশকৃত কর্মপ্রবাহ প্রতিটি ফিচারের জন্য:

```
Feature নির্বাচন → Implement (কোড লেখা) → Run (npm start) → Test  
(চেকলিস্ট মেলানো) → Debug (প্রয়োজনে) → Commit (Git-এ সংরক্ষণ)
```

Git কখন ব্যবহার করবেন

প্রতিটি বড় ফিচার সম্পূর্ণ ও টেস্ট হওয়ার পর একটি commit তৈরি করা ভালো অভ্যাস — এতে কোনো ধাপে সমস্যা হলে আগের কাজের অবস্থায় ফিরে যাওয়া সহজ হয়।

প্রথমবার Git চালু করতে:

```
git init  
git add .  
git commit -m "Initial Electron calculator setup"
```

পরবর্তী কমিটগুলোর উদাহরণ

```
git add .  
git commit -m "Add basic arithmetic calculation engine"  
  
git add .  
git commit -m "Add scientific functions: sqrt, square, log, pi"  
  
git add .  
git commit -m "Add trigonometry with DEG/RAD support"  
  
git add .  
git commit -m "Add history system with localStorage  
persistence"  
  
git add .  
git commit -m "Add clipboard copy/paste with sanitization"  
  
git add .  
git commit -m "Add dark/light theme toggle"  
  
git add .  
git commit -m "Add Web Audio API sound feedback"
```

```
git add .  
git commit -m "Add full keyboard support"
```

পরামর্শ: `.gitignore` ফাইলে `node_modules/` এবং `dist/` যোগ করুন, যাতে এই বড় ফোল্ডারগুলো Git রিপোজিটরিতে অপ্রয়োজনে যোগ না হয়:

```
node_modules/  
dist/
```

অধ্যায় ২৮ — চূড়ান্ত প্রজেক্ট স্ট্রাকচার

সম্পূর্ণ গাইড অনুসরণ করার পর আপনার প্রজেক্ট ফোল্ডারের কাঠামো এমন দেখাবে:

```
electron-scientific-calculator/
├── package.json
├── package-lock.json
├── main.js
├── preload.js
├── .gitignore
├── renderer/
│   ├── index.html
│   ├── styles.css
│   └── renderer.js
├── node_modules/           (npm install দ্বারা স্বয়ংক্রিয়ভাবে তৈরি)
└── dist/                   (প্যাকেজিং করলে তৈরি হবে — অধ্যায় ৩১)
```

অধ্যায় ২৯ — চূড়ান্ত সম্পূর্ণ সোর্স কোড

এই অধ্যায়ে প্রতিটি ফাইলের **সম্পূর্ণ, চূড়ান্ত সংস্করণ** দেওয়া হলো। আগের অধ্যায়গুলোতে ধাপে ধাপে যা তৈরি করেছেন, তার সাথে এই কোড মিলিয়ে দেখুন — যদি কোনো পার্থক্য পান, এই সংস্করণটিকেই চূড়ান্ত হিসেবে গণ্য করুন।

package.json

```
{
  "name": "electron-scientific-calculator",
  "version": "1.0.0",
  "description": "A scientific calculator built with Electron",
  "main": "main.js",
  "scripts": {
    "start": "electron .",
    "package-win": "electron-packager . ScientificCalculator --platform=win32 --arch=x64 --out=dist --overwrite",
    "package-linux": "electron-packager . ScientificCalculator --platform=linux --arch=x64 --out=dist --overwrite"
  },
  "author": "",
  "license": "ISC",
  "devDependencies": {
    "electron": "^31.0.0",
    "electron-packager": "^17.1.2"
  }
}
```

main.js

```
const { app, BrowserWindow, Menu } = require('electron');
const path = require('path');

function createWindow() {
  const mainWindow = new BrowserWindow({
    width: 380,
    height: 680,
    minWidth: 340,
    minHeight: 560,
    title: 'Scientific Calculator',
    webPreferences: {
      preload: path.join(__dirname, 'preload.js'),
    }
  });
}
```

```

    contextIsolation: true,
    nodeIntegration: false
  }
});

Menu.setApplicationMenu(null);

mainWindow.loadFile(path.join(__dirname, 'renderer',
'index.html'));
}

app.whenReady().then(() => {
  createWindow();

  app.on('activate', () => {
    if (BrowserWindow.getAllWindows().length === 0) {
      createWindow();
    }
  });
});

app.on('window-all-closed', () => {
  if (process.platform !== 'darwin') {
    app.quit();
  }
});

```

preload.js

```

const { contextBridge, clipboard } = require('electron');

contextBridge.exposeInMainWorld('electronAPI', {
  copyToClipboard: (text) => clipboard.writeText(text),
  readFromClipboard: () => clipboard.readText()
});

```

renderer/index.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>Scientific Calculator</title>
  <link rel="stylesheet" href="styles.css" />
</head>
<body>

```

```

<div class="app" id="app">
  <div class="top-bar">
    <button id="themeToggle" class="icon-btn" title="Toggle
theme">🌙</button>
    <span class="angle-mode" id="angleMode">DEG</span>
    <button id="muteToggle" class="icon-btn" title="Mute">🔊</button>
    <button id="historyToggle" class="icon-btn"
title="History">🕒</button>
  </div>

  <div class="display">
    <div class="prev-operand" id="prevOperand"></div>
    <div class="current-operand" id="currentOperand">0</div>
  </div>

  <div class="keypad">
    <button class="btn function" data-action="clear">C</button>
    <button class="btn function" data-action="deg-rad">DEG</button>
    <button class="btn function" data-action="sqrt">√x</button>
    <button class="btn operator" data-action="divide">÷</button>

    <button class="btn function" data-action="square">x²</button>
    <button class="btn function" data-action="power">x^y</button>
    <button class="btn function" data-action="log">log</button>
    <button class="btn operator" data-action="multiply">x</button>

    <button class="btn number" data-number="7">7</button>
    <button class="btn number" data-number="8">8</button>
    <button class="btn number" data-number="9">9</button>
    <button class="btn operator" data-action="subtract">-</button>

    <button class="btn number" data-number="4">4</button>
    <button class="btn number" data-number="5">5</button>
    <button class="btn number" data-number="6">6</button>
    <button class="btn operator" data-action="add">+</button>

    <button class="btn number" data-number="1">1</button>
    <button class="btn number" data-number="2">2</button>
    <button class="btn number" data-number="3">3</button>
    <button class="btn function" data-action="sin">sin</button>
  </div>
</div>

```

```

button>
    <button class="btn number zero" data-number="0">0</
button>
    <button class="btn number" data-number=".">.</button>
    <button class="btn function" data-action="cos">cos</
button>
    <button class="btn function" data-action="tan">tan</
button>

    <button class="btn function" data-action="pi"> $\pi$ </button>
    <button class="btn function" data-action="copy">copy</
button>
    <button class="btn equals" data-action="equals">=</
button>
    </div>
</div>

<div class="history-panel" id="historyPanel">
    <div class="history-header">
        <span>History</span>
        <button id="closeHistory" class="icon-btn">x</button>
    </div>
    <div class="history-list" id="historyList"></div>
    <button id="clearHistory" class="btn function full-
width">Clear History</button>
</div>

<script src="renderer.js"></script>
</body>
</html>

```

renderer/styles.css

```

:root {
    --bg: #1e1e2e;
    --app-bg: #292a3e;
    --display-bg: #20212f;
    --text: #f4f4f8;
    --btn-number-bg: #3a3b52;
    --btn-function-bg: #4b4d6b;
    --btn-operator-bg: #f0883e;
    --btn-equals-bg: #4caf7d;
    --btn-active: #5a5c7f;
    --border: #45465e;
}

body.light {

```

```

--bg: #eef0f5;
--app-bg: #ffffff;
--display-bg: #f4f5f9;
--text: #202124;
--btn-number-bg: #e8e9f0;
--btn-function-bg: #d8dae5;
--btn-operator-bg: #f0883e;
--btn-equals-bg: #4caf7d;
--btn-active: #c7c9d8;
--border: #d0d2dd;
}

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  background: var(--bg);
  font-family: 'Segoe UI', Arial, sans-serif;
  color: var(--text);
  display: flex;
  justify-content: center;
  transition: background 0.25s ease, color 0.25s ease;
}

.app {
  width: 100%;
  max-width: 380px;
  background: var(--app-bg);
  padding: 16px;
  border-radius: 12px;
  margin-top: 16px;
  transition: background 0.25s ease;
}

.top-bar {
  display: flex;
  align-items: center;
  justify-content: space-between;
  margin-bottom: 8px;
}

.icon-btn {
  background: none;
  border: none;
  font-size: 16px;
  cursor: pointer;
  color: var(--text);
}

```

```

.angle-mode {
  font-size: 12px;
  opacity: 0.7;
}

.display {
  background: var(--display-bg);
  border-radius: 8px;
  padding: 16px 12px;
  text-align: right;
  margin-bottom: 12px;
  min-height: 72px;
  transition: background 0.25s ease;
}

.prev-operand {
  font-size: 14px;
  opacity: 0.6;
  min-height: 18px;
}

.current-operand {
  font-size: 32px;
  word-wrap: break-word;
  word-break: break-all;
}

.keypad {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 8px;
}

.btn {
  border: none;
  border-radius: 8px;
  padding: 16px 0;
  font-size: 16px;
  cursor: pointer;
  color: var(--text);
  background: var(--btn-number-bg);
  transition: background 0.15s ease, transform 0.05s ease;
}

.btn.function {
  background: var(--btn-function-bg);
}

.btn.operator {

```

```

background: var(--btn-operator-bg);
color: #fff;
}

.btn.equals {
background: var(--btn-equals-bg);
color: #fff;
grid-column: span 2;
}

.btn.zero {
grid-column: span 1;
}

.btn:hover {
filter: brightness(1.08);
}

.btn:active,
.btn.pressed {
background: var(--btn-active);
transform: scale(0.96);
}

.full-width {
width: 100%;
}

.history-panel {
position: fixed;
top: 0;
right: -320px;
width: 300px;
height: 100%;
background: var(--app-bg);
border-left: 1px solid var(--border);
padding: 16px;
transition: right 0.25s ease;
overflow-y: auto;
}

.history-panel.open {
right: 0;
}

.history-header {
display: flex;
justify-content: space-between;
align-items: center;
margin-bottom: 12px;

```

```

    font-weight: bold;
}

.history-item {
    background: var(--display-bg);
    border-radius: 6px;
    padding: 8px;
    margin-bottom: 8px;
    cursor: pointer;
    font-size: 13px;
}

.history-item .expr {
    opacity: 0.6;
}

.history-item .result {
    font-size: 16px;
}

@media (max-width: 400px) {
    .app {
        max-width: 100%;
        border-radius: 0;
    }
}

```

renderer/renderer.js

```

// ===== STATE =====
let currentInput = '0';
let previousInput = '';
let operator = null;
let angleMode = 'DEG';
let isMuted = false;
let resultJustShown = false;

const currentOperandEl =
document.getElementById('currentOperand');
const prevOperandEl = document.getElementById('prevOperand');
const angleModeEl = document.getElementById('angleMode');
const historyPanel = document.getElementById('historyPanel');
const historyList = document.getElementById('historyList');

// ===== MATH HELPERS =====
function toRadians(value) {
    return angleMode === 'DEG' ? (value * Math.PI) / 180 : value;
}

```

```

function cleanFloat(num) {
    return Math.abs(num) < 1e-10 ? 0 : Math.round(num * 1e12) /
1e12;
}

function safeTrig(fn, value) {
    const rad = toRadians(value);
    if (fn === 'tan' && Math.abs(Math.cos(rad)) < 1e-10) return
'Error';
    const result = fn === 'sin' ? Math.sin(rad) : fn === 'cos' ?
Math.cos(rad) : Math.tan(rad);
    return cleanFloat(result);
}

function calculate(a, op, b) {
    a = parseFloat(a);
    b = parseFloat(b);
    switch (op) {
        case '+': return a + b;
        case '-': return a - b;
        case '*': return a * b;
        case '/': return b === 0 ? 'Error' : a / b;
        case '^': {
            const result = Math.pow(a, b);
            return isFinite(result) ? result : 'Error';
        }
        default: return b;
    }
}

// ===== DISPLAY =====
function updateDisplay() {
    currentOperandEl.textContent = currentInput;
    prevOperandEl.textContent = previousInput && operator ? ` $
{previousInput} ${operator}` : '';
    angleModeEl.textContent = angleMode;
}

function setError() {
    currentInput = 'Error';
    previousInput = '';
    operator = null;
    resultJustShown = true;
    updateDisplay();
}

// ===== INPUT HANDLERS =====
function inputNumber(num) {
    if (currentInput === 'Error' || resultJustShown) {

```

```

    currentInput = '';
    resultJustShown = false;
  }
  if (num === '.' && currentInput.includes('.')) return;
  if (currentInput === '0' && num !== '.') {
    currentInput = num;
  } else {
    currentInput = currentInput === '' ? num : currentInput +
num;
  }
  playSound('number');
  updateDisplay();
}

function chooseOperator(op) {
  if (currentInput === 'Error') return;
  if (previousInput !== '' && operator && !resultJustShown) {
    previousInput = calculate(previousInput, operator,
currentInput).toString();
  } else {
    previousInput = currentInput;
  }
  operator = op;
  currentInput = '0';
  resultJustShown = false;
  playSound('operator');
  updateDisplay();
}

function equals() {
  if (operator === null || currentInput === 'Error') return;
  const result = calculate(previousInput, operator,
currentInput);
  const expression = `${previousInput} ${operator} $
{currentInput}`;

  if (result === 'Error') {
    setError();
    playSound('error');
    return;
  }

  addToHistory(expression, result.toString());
  currentInput = result.toString();
  previousInput = '';
  operator = null;
  resultJustShown = true;
  playSound('result');
  updateDisplay();
}

```

```

function clearAll() {
  currentInput = '0';
  previousInput = '';
  operator = null;
  resultJustShown = false;
  updateDisplay();
}

// ===== SCIENTIFIC FUNCTIONS =====
function applyUnary(action) {
  if (currentInput === 'Error') return;
  const value = parseFloat(currentInput);
  let result;
  let expression;

  switch (action) {
    case 'sqrt':
      result = value < 0 ? 'Error' : Math.sqrt(value);
      expression = `sqrt(${value})`;
      break;
    case 'square':
      result = value * value;
      expression = `${value}^2`;
      break;
    case 'log':
      result = value <= 0 ? 'Error' : Math.log10(value);
      expression = `log(${value})`;
      break;
    case 'sin':
      result = safeTrig('sin', value);
      expression = `sin(${value}°/rad)`;
      break;
    case 'cos':
      result = safeTrig('cos', value);
      expression = `cos(${value}°/rad)`;
      break;
    case 'tan':
      result = safeTrig('tan', value);
      expression = `tan(${value}°/rad)`;
      break;
    default:
      return;
  }

  if (result === 'Error') {
    setError();
    playSound('error');
    return;
  }

```

```

    addToHistory(expression, result.toString());
    currentInput = result.toString();
    resultJustShown = true;
    playSound('result');
    updateDisplay();
}

function insertPi() {
    currentInput = Math.PI.toString();
    resultJustShown = true;
    updateDisplay();
}

function toggleAngleMode() {
    angleMode = angleMode === 'DEG' ? 'RAD' : 'DEG';
    updateDisplay();
}

// ===== HISTORY (localStorage) =====
function loadHistory() {
    const raw = localStorage.getItem('calc-history');
    return raw ? JSON.parse(raw) : [];
}

function saveHistory(history) {
    localStorage.setItem('calc-history',
    JSON.stringify(history));
}

function addToHistory(expression, result) {
    const history = loadHistory();
    history.unshift({ expression, result });
    if (history.length > 50) history.pop();
    saveHistory(history);
    renderHistory();
}

function renderHistory() {
    const history = loadHistory();
    historyList.innerHTML = '';
    history.forEach((item) => {
        const div = document.createElement('div');
        div.className = 'history-item';
        div.innerHTML = `<div class="expr">${item.expression}</
div><div class="result">= ${item.result}</div>`;
        div.addEventListener('click', () => {
            currentInput = item.result;
            resultJustShown = true;
            updateDisplay();

```

```

    });
    historyList.appendChild(div);
  });
}

document.getElementById('clearHistory').addEventListener('click',
() => {
  saveHistory([]);
  renderHistory();
});

document.getElementById('historyToggle').addEventListener('click',
() => {
  historyPanel.classList.toggle('open');
});

document.getElementById('closeHistory').addEventListener('click',
() => {
  historyPanel.classList.remove('open');
});

// ===== CLIPBOARD =====
async function copyResult() {
  await window.electronAPI.copyToClipboard(currentInput);
  flashCopyFeedback();
}

function flashCopyFeedback() {
  const original = currentOperandEl.style.color;
  currentOperandEl.style.color = '#4caf7d';
  setTimeout(() => {
    currentOperandEl.style.color = original;
  }, 200);
}

function sanitizeNumeric(text) {
  const cleaned = text.replace(/,/g, '').trim();
  return /^-?\d*\.\?\d+$/.test(cleaned) ? cleaned : null;
}

async function pasteValue() {
  const text = await window.electronAPI.readFromClipboard();
  const sanitized = sanitizeNumeric(text);
  if (sanitized === null) return;
  currentInput = sanitized;
  resultJustShown = false;
  updateDisplay();
}

// ===== THEME =====

```

```

function loadTheme() {
  const saved = localStorage.getItem('calc-theme') || 'dark';
  document.body.classList.toggle('light', saved === 'light');
  document.getElementById('themeToggle').textContent = saved
  === 'light' ? '☀️' : '🌙';
}

function toggleTheme() {
  const isLight = document.body.classList.toggle('light');
  localStorage.setItem('calc-theme', isLight ? 'light' :
'dark');
  document.getElementById('themeToggle').textContent =
isLight ? '☀️' : '🌙';
}

document.getElementById('themeToggle').addEventListener('click',
toggleTheme);

// ===== WEB AUDIO =====
let audioCtx = null;

function getAudioContext() {
  if (!audioCtx) {
    audioCtx = new (window.AudioContext ||
window.webkitAudioContext)();
  }
  return audioCtx;
}

function playSound(kind) {
  if (isMuted) return;
  const ctx = getAudioContext();
  const oscillator = ctx.createOscillator();
  const gain = ctx.createGain();

  const frequencies = { number: 440, operator: 550, result:
660, error: 220 };
  oscillator.frequency.value = frequencies[kind] || 440;
  oscillator.type = 'sine';

  gain.gain.setValueAtTime(0.08, ctx.currentTime);
  gain.gain.exponentialRampToValueAtTime(0.0001,
ctx.currentTime + 0.12);

  oscillator.connect(gain);
  gain.connect(ctx.destination);

  oscillator.start();
  oscillator.stop(ctx.currentTime + 0.12);
}

```

```

function loadMuteState() {
  isMuted = localStorage.getItem('calc-muted') === 'true';
  document.getElementById('muteToggle').textContent = isMuted ?
  '🔇' : '🔊';
}

function toggleMute() {
  isMuted = !isMuted;
  localStorage.setItem('calc-muted', isMuted.toString());
  document.getElementById('muteToggle').textContent = isMuted ?
  '🔇' : '🔊';
}

document.getElementById('muteToggle').addEventListener('click',
toggleMute);

// ===== BUTTON WIRING =====
document.querySelectorAll('.btn[data-number]').forEach((btn) =>
{
  btn.addEventListener('click', () =>
inputNumber(btn.dataset.number));
});

const actionMap = {
  clear: clearAll,
  'deg-rad': toggleAngleMode,
  sqrt: () => applyUnary('sqrt'),
  square: () => applyUnary('square'),
  log: () => applyUnary('log'),
  sin: () => applyUnary('sin'),
  cos: () => applyUnary('cos'),
  tan: () => applyUnary('tan'),
  pi: insertPi,
  copy: copyResult,
  equals: equals,
  add: () => chooseOperator('+'),
  subtract: () => chooseOperator('-'),
  multiply: () => chooseOperator('*'),
  divide: () => chooseOperator('/'),
  power: () => chooseOperator('^')
};

document.querySelectorAll('.btn[data-action]').forEach((btn) =>
{
  btn.addEventListener('click', () => {
    const action = actionMap[btn.dataset.action];
    if (action) action();
  });
});

```

```
// ===== KEYBOARD SUPPORT =====
const keyToSelector = {
  '+': '[data-action="add"]',
  '-': '[data-action="subtract"]',
  '*': '[data-action="multiply"]',
  '/': '[data-action="divide"]',
  '^': '[data-action="power"]',
  Enter: '[data-action="equals"]',
  '=': '[data-action="equals"]',
  Escape: '[data-action="clear"]',
  s: '[data-action="sin"]',
  c: '[data-action="cos"]',
  t: '[data-action="tan"]',
  r: '[data-action="sqrt"]',
  q: '[data-action="square"]',
  p: '[data-action="pi"]',
  l: '[data-action="log"]',
  d: '[data-action="deg-rad"]'
};

function pressVisual(selector) {
  const el = document.querySelector(selector);
  if (el) el.classList.add('pressed');
}

function releaseVisual(selector) {
  const el = document.querySelector(selector);
  if (el) el.classList.remove('pressed');
}

document.addEventListener('keydown', (e) => {
  if (/^[0-9]$/.test(e.key) || e.key === '.') {
    inputNumber(e.key);
    return;
  }

  if (e.key === 'h') {
    historyPanel.classList.toggle('open');
    return;
  }

  const selector = keyToSelector[e.key];
  if (selector) {
    pressVisual(selector);
    const el = document.querySelector(selector);
    if (el) el.click();
  }

  if ((e.ctrlKey || e.metaKey) && e.key.toLowerCase() === 'c')
```

```
{
  copyResult();
}
if ((e.ctrlKey || e.metaKey) && e.key.toLowerCase() === 'v')
{
  pasteValue();
}
});

document.addEventListener('keyup', (e) => {
  const selector = keyToButtonSelector[e.key];
  if (selector) releaseVisual(selector);
});

// ===== INIT =====
loadTheme();
loadMuteState();
renderHistory();
updateDisplay();
```

অধ্যায় ৩০ — চূড়ান্ত রান নির্দেশনা

সব ফাইল উপরের সংস্করণের সাথে মিলিয়ে নেওয়ার পর, প্রজেক্ট ফোল্ডারে থেকে টার্মিনালে চালান:

```
npm start
```

প্রত্যাশিত ফলাফল: একটি ৩৮০x৬৮০ আকারের উইন্ডো খুলবে, টাইটেল "Scientific Calculator", ডার্ক থিমে (প্রথমবার) শুরু হবে, সব বাটন কার্যকর থাকবে, এবং অধ্যায় ২৩-এর সম্পূর্ণ টেস্টিং চেকলিস্ট পাস করবে।

অধ্যায় ৩১ — প্যাকেজিং: ডিস্ট্রিবিউটেবল অ্যাপ তৈরি করা

ডেভেলপমেন্ট শেষে, আমাদের Electron অ্যাপকে একটি স্বতন্ত্র (standalone) ডেস্কটপ প্রোগ্রামে পরিণত করা যায়, যা Node.js/npm ইনস্টল না থাকা কম্পিউটারেও চলবে।

কেন electron-packager দরকার

`electron-packager` আমাদের সোর্স কোড, Electron রানটাইম, এবং Node.js — সবকিছু একসাথে বান্ডিল করে একটি চালানোর-উপযোগী (executable) ফোল্ডার তৈরি করে দেয়।

ইনস্টলেশন

অধ্যায় ৫-এই এটি `devDependency` হিসেবে ইনস্টল করা হয়েছে:

```
npm install --save-dev electron-packager
```

কনফিগারেশন

`package.json`-এর `scripts` অংশে ইতিমধ্যে দুটি কমান্ড যোগ করা আছে (অধ্যায় ৪ দেখুন):

```
"package-win": "electron-packager . ScientificCalculator --  
platform=win32 --arch=x64 --out=dist --overwrite",  
"package-linux": "electron-packager . ScientificCalculator --  
platform=linux --arch=x64 --out=dist --overwrite"
```

Windows-এর জন্য বিন্ড করা

```
npm run package-win
```

আউটপুট ফোল্ডার: `dist/ScientificCalculator-win32-x64/` — এর ভেতরে `ScientificCalculator.exe` থাকবে, যা ডাবল-ক্লিক করলেই চলবে।

Linux-এর জন্য বিল্ড করা

```
npm run package-linux
```

আউটপুট ফোল্ডার: `dist/ScientificCalculator-linux-x64/` — এর ভেতরে `ScientificCalculator` নামে একটি এক্সিকিউটেবল ফাইল থাকবে। রান করতে:

```
cd dist/ScientificCalculator-linux-x64  
./ScientificCalculator
```

নোট: ক্রস-প্ল্যাটফর্ম প্যাকেজিং সম্ভব (যেমন Linux থেকে Windows-এর জন্য বিল্ড করা), তবে সবচেয়ে নির্ভরযোগ্য ফলাফলের জন্য যে অপারেটিং সিস্টেমে চালানো হবে, সেই একই অপারেটিং সিস্টেম থেকে বিল্ড করা ভালো — বিশেষত macOS-এর জন্য কোড সাইনিং প্রয়োজন হতে পারে।

সাধারণ সমস্যা: প্যাকেজিং কমান্ড ব্যর্থ হলে, `node_modules` ফোল্ডারটি ঠিকভাবে ইনস্টল করা আছে কিনা এবং `--overwrite` ফ্ল্যাগ ব্যবহার করছেন কিনা (আগের বিল্ড থাকলে) যাচাই করুন।

অধ্যায় ৩২ — চূড়ান্ত প্রজেক্ট চেকলিস্ট

- ☐ Node.js ইনস্টল করা হয়েছে
- ☐ npm দিয়ে প্রজেক্ট ইনিশিয়ালাইজ করা হয়েছে
- ☐ Electron ইনস্টল করা হয়েছে
- ☐ Main Process (`main.js`) কাজ করছে
- ☐ Preload (`preload.js`) কাজ করছে
- ☐ নিরাপদ Context Isolation চালু আছে
- ☐ বেসিক ক্যালকুলেটর কাজ করছে
- ☐ বৈজ্ঞানিক ফাংশন কাজ করছে
- ☐ DEG/RAD কাজ করছে
- ☐ History কাজ করছে
- ☐ localStorage persistence কাজ করছে
- ☐ Copy কাজ করছে
- ☐ Paste কাজ করছে
- ☐ Theme কাজ করছে
- ☐ Audio কাজ করছে
- ☐ কীবোর্ড শর্টকাট কাজ করছে
- ☐ এরর হ্যান্ডলিং কাজ করছে
- ☐

নিরাপত্তা যাচাই করা হয়েছে

☐

অ্যাপ্লিকেশন সম্পূর্ণভাবে টেস্ট করা হয়েছে

☐

অ্যাপ্লিকেশন প্যাকেজ করা হয়েছে

আপনার Electron Scientific Calculator সম্পূর্ণভাবে তৈরি, পরীক্ষা এবং package করা হয়েছে।